

THE UNIVERSITY OF LAHORE

Final Year Project Automation System (FYPAS)

Submitted by

Aamna Arif (70124659) Shah Nawaz (70126028)

Saim Ghafoor (70126998)

Project Supervisor:

Mr. Abdul Dayyan

BACHELOR OF SCIENCE IN SOFTWARE ENGINEERING

DEPARTMENT OF SOFTWARE ENGINEERING, UNIVERSITY OF LAHORE

Table of Contents

Final Year Project Automation System (FYPAS) i
ABSTRACT vi
DEDICATION vii
ACKNOWLEDGMENT viii
Chapter 1: 1
Introduction to the Problem 1
1.1 Introduction 2
1.2 Problem Statement 2
1.3 Scope 2
1.4 Relevance or Importance of Problem 2
1.5 Background 2
1.6 Previous Related Work 2
1.7 Competitive Analysis 3
1.7.1 Trello 3
1.7.2 Slack 3
1.7.3 Asana 3
1.7.4 Supervisor Connect 3
1.7.5 Student Management Systems (SMS) 3
1.8 Objectives 3
1.9 Motivation 4
1.10 Requirements 4
1.10.1 Functional Requirements 4
1.10.2 Non-Functional Requirements 5
1.11 Technological Requirements 5
1.12 Features of Project 6
1.12.1 Student Portal 6
1.12.2 Supervisor Portal 6
1.12.3 Coordinator/Admin Portal 6
1.12.4 System-Wide Features 6
1.13 Architectural Design 6
1.13.1 Hardware Components 6

1.13.2 Software Components 6
1.13.3 Network Components 7
1.14 Implementation Tools and Techniques 7
1.14.1 Frontend 7
1.14.2 Backend 7
1.14.3 Database 7
1.14.4 Version Control 7
1.14.5 Server and Hosting 7
1.14.6 Security 7
1.14.7 API Integration 7
1.14.8 Testing 8
1.14.9 Deployment 8
1.14.10 UI Design 8
1.14.11 Collaboration 8
1.15 Project Plan 8
1.15.1 Duration 8
1.15.2 Milestones 8
1.16 Version Control 9
Software Requirement Specification 10
2.1 Introduction 11
2.1.1 Purpose 11
2.1.2 Scope 11
2.1.3 Definitions, Acronyms, and Abbreviations 11
2.1.4 References 11
2.1.5 Overview 11
2.2 Overall Description 11
2.2.1 Product Perspective 11
2.2.2 User Interfaces 11
2.2.3 Hardware Interfaces 11
2.2.4 Software Interfaces 12
2.2.5 Communications Interfaces 12
2.2.6 Memory 12

2.2.7 Operations....12
2.2.8 Site Adaptation Requirements....12
2.2.9 Product Functions....12
2.2.10 User Characteristics....13
2.2.11 Constraints....13
2.2.12 Assumptions and Dependencies....13
2.2.13 Apportioning of Requirements....13
2.3 Specific Requirements....13
2.3.1 Functional Requirements....13
2.3.2 Non-Functional Requirements....13
2.4 Appendixes....14
2.5 Index....14
Chapter 3:....15
Use Case Analysis....15
3.1 Functional Requirement Use Case....16
3.1.1 Use Case 1: Student Portal....16
3.1.2 Use Case 2: Supervisor Portal....19
3.1.3 Use Case 3: Coordinator Portal....22
Chapter 4:....26
Design....26
4.1 Architecture Diagram of System....27
4.2 ERD Diagram of System....27
4.3 DFD Diagram Level 1....27
4.4 DFD Diagram Level 2....28
4.5 Class Diagram of System....29
4.6 Activity Diagram:....30
4.6.1 Student Portal:....30
4.6.2 Supervisor Portal:....36
4.6.3 Coordinator Portal:....40
4.7 Sequence Diagram....44
4.7.1 Student Portal....44
4.7.2 Supervisor Portal:....47

4.7.3 Coordinator Portal: 49
4.8 Collaboration Diagram 51
4.8.1 Student Portal: 51
4.8.2 Supervisor Portal:.....54
4.8.3 Coordinator Portal: 56
4.9 State Transition Diagram.....58
4.10 Component Diagram of System 59
4.11 Deployment Diagram 60
References.....63

ABSTRACT

The Final Year Project Automation System (FYPAS) is a web-based platform designed to streamline the management of final year projects at the University of Lahore. It automates proposal submission, supervisor matching, progress tracking, and communication, addressing inefficiencies in the manual system. Built with React, Django, and SQLite, FYPAS integrates GitHub for progress monitoring and Google Scholar for research references. Feature schema It ensures idea originality, enforces deadlines, and provides real-time notifications, enhancing collaboration and transparency. With a user-friendly interface and robust security, FYPAS transforms the FYP process, improving efficiency and student-supervisor coordination.

DEDICATION

We dedicate this Final Year Project Automation System (FYPAS) to our families, whose unwavering support and encouragement have been our foundation throughout this journey. To our esteemed supervisor, Mr. Abdul Dayyan, for his invaluable guidance and mentorship. To the faculty and staff of the University of Lahore, whose knowledge and inspiration fueled our pursuit of excellence. And to our peers, whose collaboration and camaraderie made this endeavor a rewarding experience. This project reflects our collective commitment to advancing education through technology.

ACKNOWLEDGMENT

We express our deepest gratitude to all those who have contributed to the successful completion of the Final Year Project Automation System (FYPAS). This project would not have been possible without the guidance, support, and encouragement of numerous individuals and institutions.

First and foremost, we extend our heartfelt thanks to our project supervisor, **Mr. Abdul Dayyan**, for his unwavering guidance, insightful feedback, and constant encouragement throughout the project. His expertise and mentorship were instrumental in shaping our vision and ensuring the project's success.

We are immensely grateful to the **University of Lahore**, particularly the **Department of Software Engineering**, for providing us with the resources, academic environment, and opportunities to undertake this ambitious project. The faculty and staff have been a constant source of inspiration and support, fostering an environment conducive to innovation and learning.

We would also like to thank our families for their endless love, patience, and encouragement. Their unwavering support provided us with the strength to overcome challenges and stay focused throughout this journey.

Our gratitude extends to our peers and colleagues, whose collaboration, constructive feedback, and camaraderie made this project a rewarding experience. Their insights and teamwork were invaluable in refining our ideas and implementation.

Finally, we acknowledge the open-source community and the developers of tools such as **React**, **Django**, **SQLite**, **GitHub**, and others, whose technologies formed the backbone of FYPAS. Their contributions to the global tech ecosystem enabled us to build a robust and efficient system.

Chapter 1:

Introduction to the Problem

1.1 Introduction

The Final Year Project Automation System (FYPAS) is a web-based platform designed to streamline the management of final year projects (FYPs) at the University of Lahore. By automating critical processes such as proposal submission, supervisor matching, progress tracking, and communication, FYPAS addresses inefficiencies in the current manual system. This chapter provides an overview of the problem, its significance, objectives, and technological requirements of the system.

1.2 Problem Statement

The existing FYP management system at the University of Lahore relies on manual processes, resulting in inefficiencies, duplicated efforts, and communication barriers. These issues lead to delays, reduced productivity, and frustration among students, supervisors, and coordinators, necessitating an automated solution.

1.3 Scope

FYPAS automates the entire FYP lifecycle, encompassing proposal submission, idea duplication checks, supervisor allocation, progress monitoring, and real-time communication. Targeted at students, supervisors, and coordinators at the University of Lahore, the system is scalable for broader institutional use. It integrates GitHub for progress tracking and Google Scholar for research references, ensuring a comprehensive and modern approach.

1.4 Relevance or Importance of Problem

Effective FYP management is essential for students to apply academic knowledge practically, fostering innovation and professional skills. The manual system's inefficiencies—delays, duplicated efforts, and communication gaps—hinder productivity and increase frustration. FYPAS aims to streamline processes, enhance collaboration, and reduce administrative burdens, thereby improving the academic experience and project outcomes.

1.5 Background

The University of Lahore employs a manual FYP system involving physical proposal submissions, manual supervisor assignments, and periodic check-ins. This approach lacks real-time communication, delays feedback, and fails to ensure idea originality, leading to inefficiencies and compromised project quality. An automated system is critical to modernize and optimize these processes.

1.6 Previous Related Work

Several global universities have implemented partial FYP automation systems. For example, a Malaysian system automated supervisor assignments and plagiarism checks but lacked progress tracking (Rahman et al., 2019). Wang et al. (2020) demonstrated that real-time tracking enhances efficiency. FYPAS builds on these efforts by integrating all FYP stages into a comprehensive platform.

1.7 Competitive Analysis

1.7.1 Trello

- **Overview**: A project management tool for task organization.
- **Features**: Task management, collaboration, progress tracking.
- **Target Audience**: General project managers, students.
- **Analysis**: Lacks FYP-specific features such as supervisor matching and plagiarism detection.

1.7.2 Slack

- **Overview**: A communication platform for team collaboration.
- **Features**: Messaging, file sharing, integrations.
- **Target Audience**: Businesses, educational institutions.
- **Analysis**: Excels in communication but lacks FYP-specific task management capabilities.

1.7.3 Asana

- **Overview**: A project management tool for task tracking.
- **Features**: Task management, timelines, collaboration.
- **Target Audience**: Companies, educational institutions.
- **Analysis**: Not tailored for academic FYP processes, limiting its applicability.

1.7.4 Supervisor Connect

- **Overview**: An academic project management platform.
- **Features**: Supervisor matching, progress tracking.
- **Target Audience**: Universities.
- **Analysis**: Lacks idea originality checks and GitHub integration, reducing its scope.

1.7.5 Student Management Systems (SMS)

- **Overview**: University administrative systems.
- **Features**: Student records, proposal submission.
- **Target Audience**: Universities.
- **Analysis**: Generalized systems not designed specifically for FYP management.

1.8 Objectives

- Implement efficient proposal management with idea duplication checks.
- Automate supervisor matching based on expertise and availability.
- Enforce deadlines through submission locks and notifications.
- Integrate GitHub for real-time progress tracking.
- Provide secure login mechanisms.
- Enable real-time communication and notifications.
- Develop a centralized coordinator dashboard.
- Ensure a user-friendly interface.
- Promote transparency and accountability.

- Offer idea reference and suggestion tools.
- Create an admin panel for user management.
- Support coordinator announcement features.
- Ensure mobile compatibility.

1.9 Motivation

- **Student Experience**: Simplifies feedback and enhances collaboration.
- **Supervisor Efficiency**: Reduces administrative workload.
- **University Reputation**: Strengthens credibility through modernized processes.
- **Time and Resource Efficiency**: Eliminates redundant tasks.
- **Real-Time Collaboration**: Facilitates timely interventions and support.

1.10 Requirements

1.10.1 Functional Requirements

1.10.1.1 Student Portal

- **Idea Submission**: Students can submit project proposals, including title, description, and domain, through a web-based portal.
- **Idea Duplication Check**: The system checks proposals against a database of past FYPs (covering 4 years) and current submissions, notifying students instantly of duplicates.
- **Supervisor Matching**: Suggests supervisors based on domain expertise and availability, allowing students to view profiles and send supervision requests.
- **Progress Tracking**: Enables students to upload progress reports, link GitHub repositories, and track milestones, ensuring supervisor visibility.
- **Notifications**: Provides real-time updates on proposal status, supervisor approvals/rejections, deadlines, and other events.

1.10.1.2 Supervisor Portal

- **Profile Management**: Supervisors can create and update profiles, specifying expertise, availability, and research interests.
- **Proposal Approval/Rejection**: Enables supervisors to review, approve, or reject proposals, providing feedback for rejections.
- **Progress Monitoring**: Allows supervisors to track student progress through reports and GitHub commits, with access to contribution details.
- **Feedback and Communication**: Provides an internal messaging tool for supervisor-student interaction.

1.10.1.3 Coordinator/Admin Portal

- **Proposal Review**: Coordinators review proposals, approve or reject them based on university standards, and notify students and supervisors.
- **Announcements**: Coordinators post announcements, including deadlines and guidelines, ensuring all users receive notifications.
- **Dashboard**: Offers a centralized dashboard to monitor project progress, supervisor allocations, and departmental metrics.

- **User Management**: Admins can add, remove, or update user accounts, managing roles and permissions.

1.10.2 Non-Functional Requirements

1.10.2.1 Usability

- Provides an intuitive, user-friendly interface for all users, minimizing the learning curve.
- Ensures straightforward navigation with clear instructions and tooltips.

1.10.2.2 Performance

- Handles at least 1000 concurrent users during peak periods with response times under 2 seconds.
- Maintains stability under high load conditions.

1.10.2.3 Security

- Ensures data privacy using SSL/TLS encryption for all communications.
- Implements secure authentication via University of Lahore Gmail accounts using OAuth.
- Enforces role-based access control to restrict unauthorized access.

1.10.2.4 Scalability

- Supports up to 3000 users within 3 years of deployment.
- Allows modular expansion for additional features without performance degradation.

1.10.2.5 Reliability

- Achieves 99.9% uptime, ensuring minimal downtime during academic semesters.
- Maintains 100% data integrity for all submissions, feedback, and records.

1.10.2.6 Backup and Recovery

- Implements automated daily backups to protect user data and submissions.
- Provides recovery mechanisms to restore data within 4 hours of system failure.

1.10.2.7 Support

- Includes a comprehensive help section with FAQs, user guides, and tutorials.

1.11 Technological Requirements

- Web platform accessible via modern browsers.
- GitHub API integration for progress tracking.
- SQLite database for data storage.
- Cloud hosting on platforms like AWS EC2.
- Integrated messaging tools for communication.

1.12 Features of Project

1.12.1 Student Portal

- Submit project title, description, and domain.
- Cross-check ideas against past FYPs for originality.
- Suggest supervisors based on expertise.
- Send supervision requests with project details.
- Upload progress reports and track milestones.
- Receive real-time notifications for approvals and deadlines.
- Access referencing tools linked to Google Scholar and journals.

1.12.2 Supervisor Portal

- Update expertise, availability, and research interests.
- Review and approve/reject proposals with feedback.
- Monitor progress via GitHub commits and reports.
- Communicate with students through internal messaging.

1.12.3 Coordinator/Admin Portal

- Review proposals for compliance with university standards.
- Post announcements for deadlines and updates.
- Monitor overall project progress and supervisor allocations.
- Manage user accounts and permissions.
- Handle system maintenance and backups.
- Send global notifications to users.

1.12.4 System-Wide Features

- GitHub integration for contribution tracking.
- Searchable repository of past FYPs.
- Deadline enforcement with submission locks.
- Notification system for key events.
- Report generation for project status summaries.
- Feedback system for post-project evaluations.

1.13 Architectural Design

1.13.1 Hardware Components

- **Server**: Hosts application and SQLite database (Apache/Nginx).
- **User Devices**: Laptops and mobiles with modern browsers.

1.13.2 Software Components

- **Web Application**: HTML, CSS, Tailwind (frontend); Django (backend).
- **Database**: SQLite for storing user data and proposals.

- **Version Control**: GitHub for code tracking.
- **Authentication**: Admin-only secure logins.

1.13.3 Network Components

- **Internet**: Requires stable connectivity.
- **Security**: SSL encryption and role-based access control.

1.14 Implementation Tools and Techniques

1.14.1 Frontend

- HTML, CSS, Tailwind for structure and responsive design.
- JavaScript/React.js for dynamic content.
- Tools: Visual Studio Code, Cursor AI.

1.14.2 Backend

- Django for robust backend framework.
- Tools: Postman, Visual Studio Code.

1.14.3 Database

- SQLite for storing user data and proposals.
- Tools: SQLite Workbench, DBeaver, Lucidchart, draw.io.

1.14.4 Version Control

- Git/GitHub for code tracking and collaboration.
- Tools: GitHub Desktop, SourceTree.

1.14.5 Server and Hosting

- Hosting: AWS EC2, Hostinger.
- Web Server: Apache/Nginx.
- Tools: FileZilla, SSH.

1.14.6 Security

- SSL/TLS for encrypted communication.
- JWT for secure authentication.
- Tools: Let’s Encrypt, JWT libraries.

1.14.7 API Integration

- GitHub API for contribution tracking.
- Google Scholar API for research references.
- Tools: RESTful APIs, Postman.

1.14.8 Testing

- Unit testing with PyTest (Python).
- User Acceptance Testing (UAT) with stakeholder feedback.
- Tools: Selenium, Postman.

1.14.9 Deployment

- CI/CD with GitHub Actions for automation.
- Containerization with Docker.
- Tools: Jenkins, Docker Compose.

1.14.10 UI Design

- Figma for wireframes and prototypes.
- Tools: Adobe XD.

1.14.11 Collaboration

- Project management with Trello, Jira.
- Communication via Slack, Zoom.

1.15 Project Plan

1.15.1 Duration

- **Start**: January 15, 2025
- **End**: December 15, 2025

1.15.2 Milestones

Milestone	Tasks	Assigned To	Duration	Dependency
Requirement Analysis	Gather requirements, finalize SRS	All Members	Jan 15–Feb 15	None
System Design	Create ERD, architecture diagram	Aamna	Feb 15–Feb 22	Requirement Analysis
Figma Design	Create Figma design	Aamna, Saim	Feb 22–Mar 15	System Design
Frontend Development	Build UI	All Members	Mar 16–May 1	Figma Design
Backend Development	Implement APIs, database	Saim, Shah Nawaz	May 16–Aug 15	System Design

Integration	Connect frontend, backend, APIs	Saim, Shah Nawaz	Aug 16–Oct 15	Development
Testing	Unit, integration, UAT	Team	Oct 16–Nov 15	Integration
Deployment	Deploy to AWS, configure CI/CD	Aamna	Nov 16–Dec 1	Testing
User Training	Create user manual, train users	Saim	Dec 2–Dec 10	Deployment

1.16 Version Control

Version ID	Proposal ID	Updated By (User ID)	Update Description	Date/Time	Action Type
1	101	70124659	Initial proposal submission	2024-12-01 12:30:46	Created

Chapter 2:

Software Requirement Specification

2.1 Introduction

2.1.1 Purpose

This Software Requirement Specification (SRS) delineates the requirements for the Final Year Project Automation System (FYPAS), targeting students, supervisors, and coordinators/admins at the University of Lahore.

2.1.2 Scope

FYPAS automates FYP management, including proposal submission, supervisor matching, progress tracking, and communication. It enhances efficiency, ensures originality, and provides a user-friendly interface.

2.1.3 Definitions, Acronyms, and Abbreviations

- **FYP**: Final Year Project
- **FYPAS**: Final Year Project Automation System
- **UOL**: University of Lahore
- **SRS**: Software Requirement Specification
- **API**: Application Programming Interface

2.1.4 References

- Project Proposal Document
- Final Year Project Manual Document
- Rahman et al., 2019
- Wang et al., 2020

2.1.5 Overview

This SRS covers product perspective, functions, user characteristics, constraints, assumptions, and specific requirements.

2.2 Overall Description

2.2.1 Product Perspective

FYPAS is a standalone web application interfacing with GitHub and Google Scholar APIs, integrated with university email authentication.

2.2.2 User Interfaces

- **Student Portal**: Proposal submission, supervisor matching, progress tracking.
- **Supervisor Portal**: Profile management, proposal review, progress monitoring.
- **Coordinator Portal/Admin**: Proposal review, announcements, dashboard, user management, system maintenance.

2.2.3 Hardware Interfaces

- Standard devices (laptops, mobiles) with browsers.
- Server with Apache/Nginx, SQLite.

2.2.4 Software Interfaces

- HTML, CSS, Tailwind, JS: Frontend framework.
- Django: Backend logic.
- SQLite: Database management.
- GitHub API: Progress tracking.
- Google Scholar API: Research references.

2.2.5 Communications Interfaces

- HTTPS for secure communication.
- WebSocket for real-time notifications.

2.2.6 Memory

- **Server**: 16GB RAM, 500GB SSD.
- **Client**: Standard browser requirements.

2.2.7 Operations

- Regular backups, system updates.
- User support via helpdesk.

2.2.8 Site Adaptation Requirements

- Configurable for university-specific domains, emails.

2.2.9 Product Functions

ID	Name	Description	Input	Output	Requirements	Basic Work Flow
FR_01	Idea Submission	Submit project proposal	Title, description, domain	Submission confirmation	Internet, email	Enter details, submit, check duplicates
FR_02	Supervisor Matching	Match students with supervisors	Project details	Supervisor suggestions	Database access	Select domain, view profiles, send request
FR_03	Progress Tracking	Upload progress reports, track milestones	Reports, GitHub commits	Updated project status	GitHub account	Upload files, update milestones

FR_04	Notifications	Send real-time alerts	Event triggers	Email, in-app notifications	Email service	Trigger event, send alert
FR_05	Proposal Review	Coordinators review proposals	Submitted proposals	Approval/rejection notice	Coordinator access	Review, approve/reject, notify

2.2.10 User Characteristics

- **Students**: BSSE students, basic technical skills.
- **Supervisors**: Faculty with domain expertise.
- **Coordinator/Admins**: IT staff, advanced technical expertise.

2.2.11 Constraints

- **Regulatory**: Compliance with university policies.
- **Hardware**: Standard server, client devices.
- **Security**: SSL, JWT authentication.
- **Reliability**: 99.9% uptime.

2.2.12 Assumptions and Dependencies

- Stable internet access.
- University email accounts for all users.
- GitHub accounts for students.

2.2.13 Apportioning of Requirements

- Future versions may include AI-based idea suggestions, advanced analytics.

2.3 Specific Requirements

2.3.1 Functional Requirements

- **Idea Submission**: Submit and check for duplicates.
- **Supervisor Matching**: Suggest based on expertise.
- **Progress Tracking**: Monitor via GitHub, reports.
- **Notifications**: Real-time alerts.
- **Proposal Review**: Coordinator approval/rejection.

2.3.2 Non-Functional Requirements

- **Usability**: Intuitive interface, minimal learning curve.
- **Reliability**: 99.9% uptime, data integrity.
- **Performance**: Handle 1000 concurrent users.
- **Portability**: Browser-based, device-agnostic.
- **Maintainability**: Modular code, easy updates.
- **License**: University-owned, open-source components.

2.4 Appendixes

- Glossary of terms.
- Sample screenshots (post-development).

2.5 Index

- Functional Requirements: 2.3.1
- Non-Functional Requirements: 2.3.2
- Architecture Diagram: 4.1

Chapter 3:

Use Case Analysis

3.1 Functional Requirement Use Case

3.1.1 Use Case 1: Student Portal

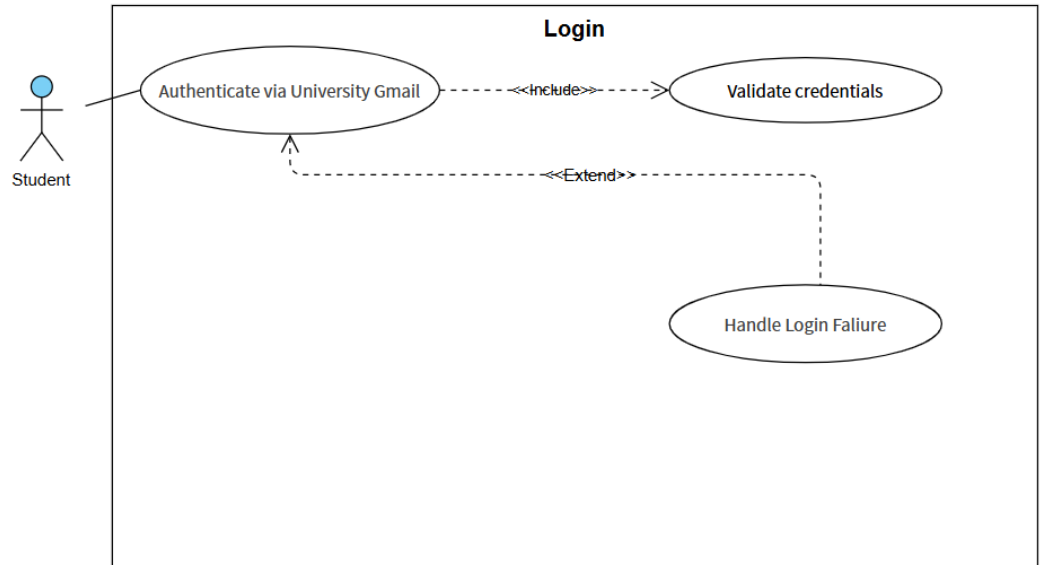


Fig 3.1.1.1. Login process for student portal

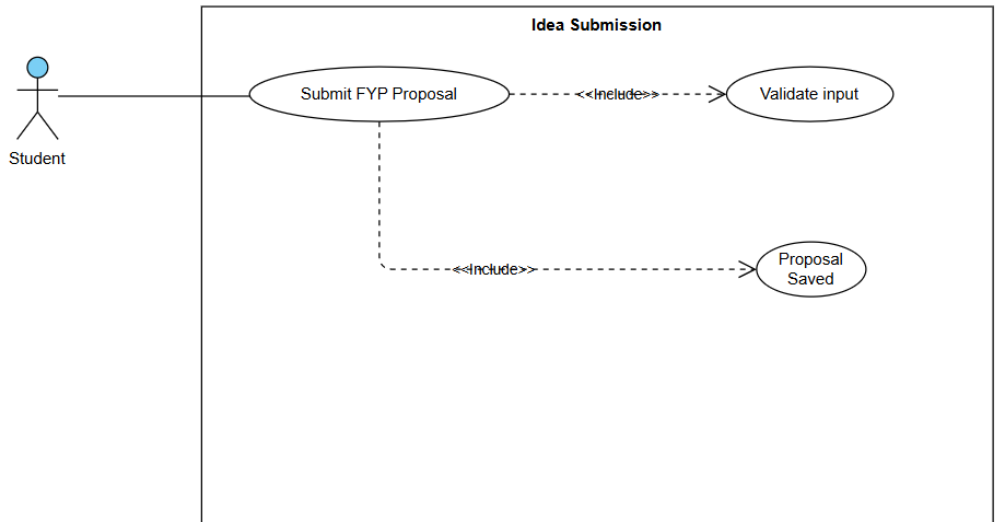


Fig 3.1.1.2. Idea submission process for student portal

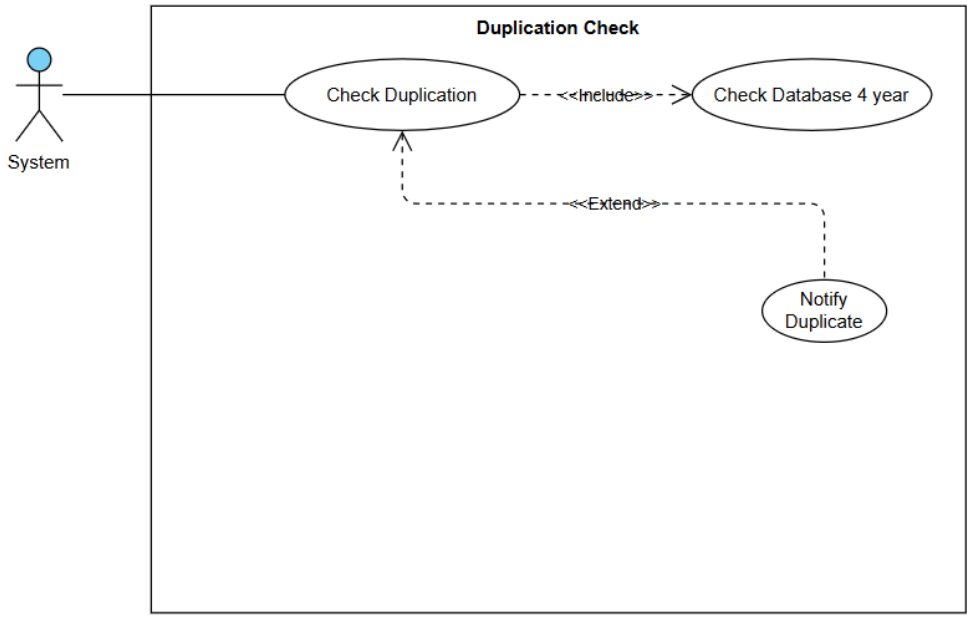


Fig 3.1.1.3. Idea duplication check process for student portal

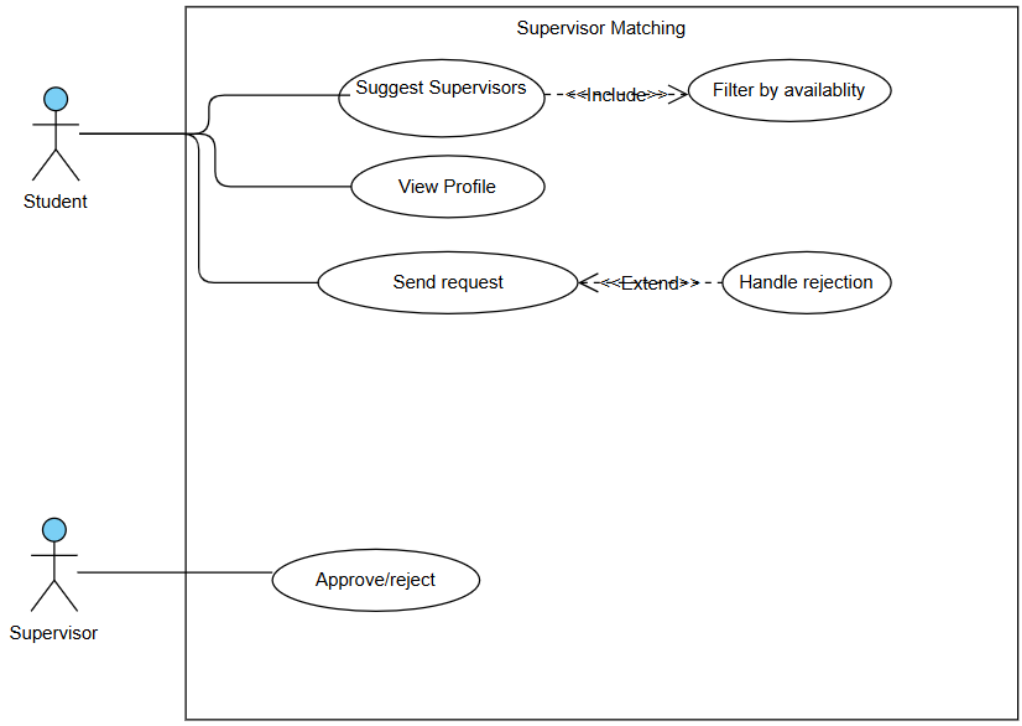


Fig 3.1.1.4. Supervisor matching process for student portal

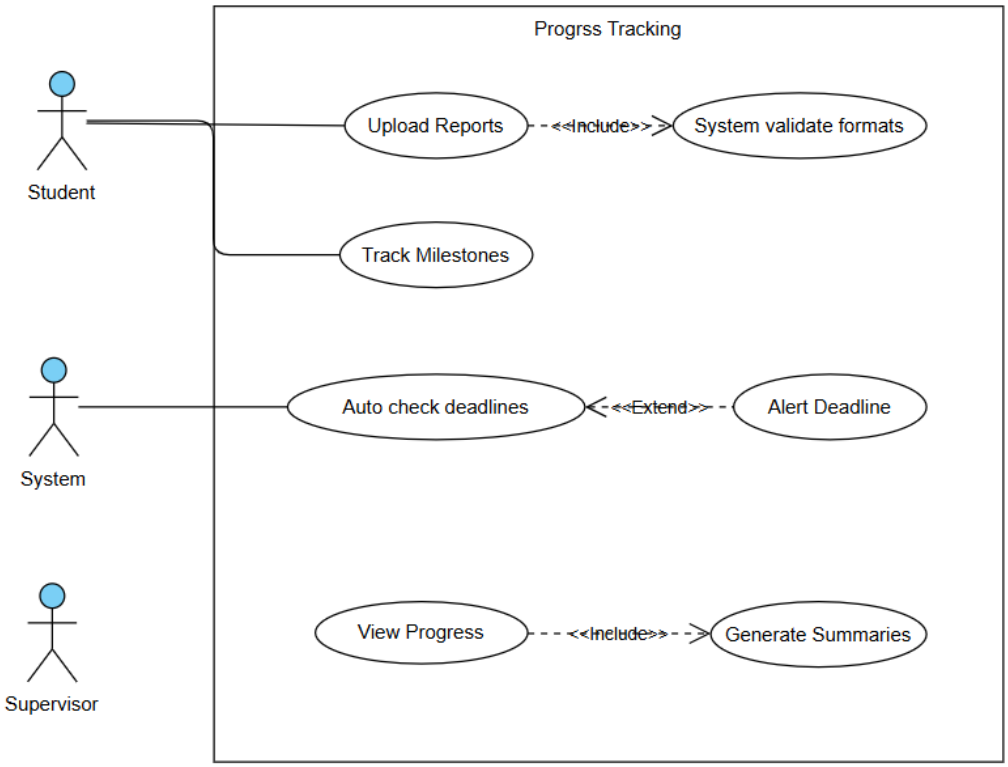


Fig 3.1.1.5. Progress tracking process for student portal

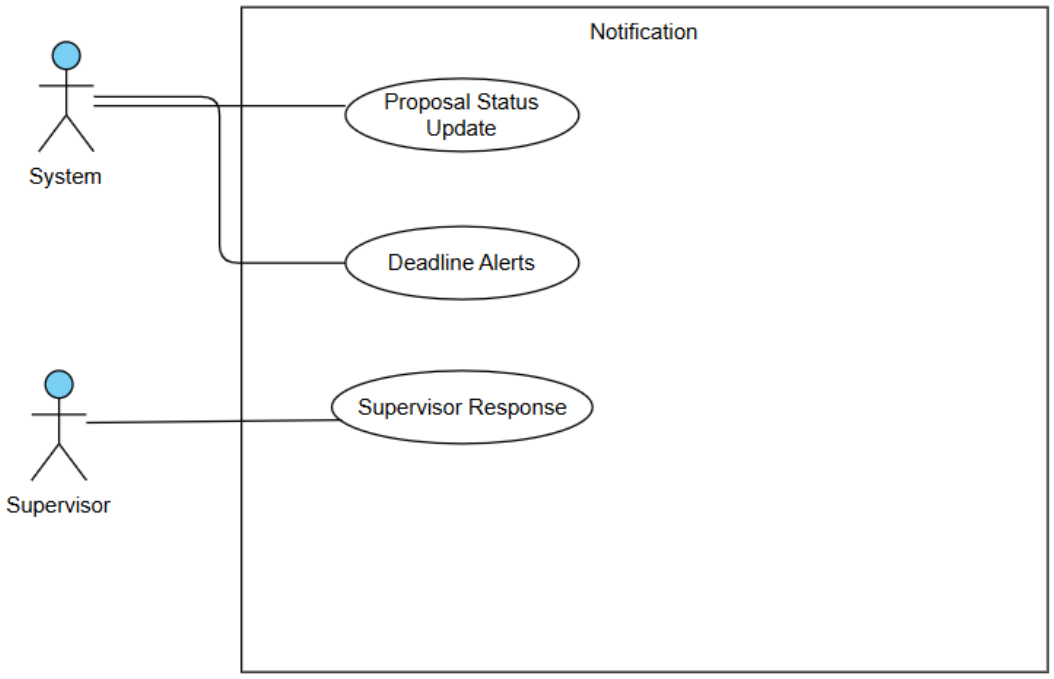


Fig 3.1.1.6. Notification process for student portal

3.1.1.2 Explained Detail for Student Portal Based System

Use Case ID	Use Case Name	Actor	Description	Preconditions	Postconditions
UC1	Student Login	Student	Student logs in using university Gmail credentials.	Student has a valid university Gmail account.	Student is authenticated and granted access to portal features.
UC2	Idea Submission	Student	Submit project proposals with title, description, and domain.	Student is logged in.	Proposal is stored and submitted for review**.**
UC3	Idea Duplication Check	System	System checks submitted ideas for duplication against current and past FYPs.	Proposal is submitted**.**	Student is notified if duplication is found or cleared to proceed.
UC4	Supervisor Matching	Student	View supervisor suggestions based on domain; send request to preferred supervisor.	Student has submitted a valid idea.	Supervisor request is sent; student can track request status.
UC5	Progress Tracking	Student	Upload progress reports, link GitHub, and track project milestones.	Supervisor is assigned.	Progress data is visible to both student and supervisor.
UC6	Notifications	System	Provide real-time updates on proposal status, supervisor responses, deadlines, and announcements.	Events or status changes occur in the system.	Student receives notifications via portal (and optionally email).

3.1.2 Use Case 2: Supervisor Portal

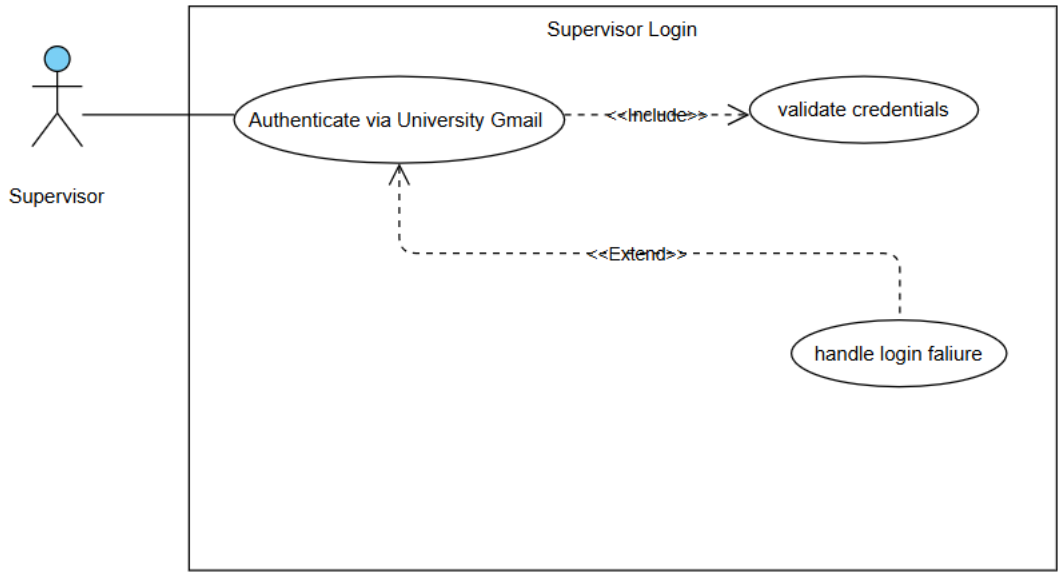


Fig 3.1.2.1. Login process for supervisor portal

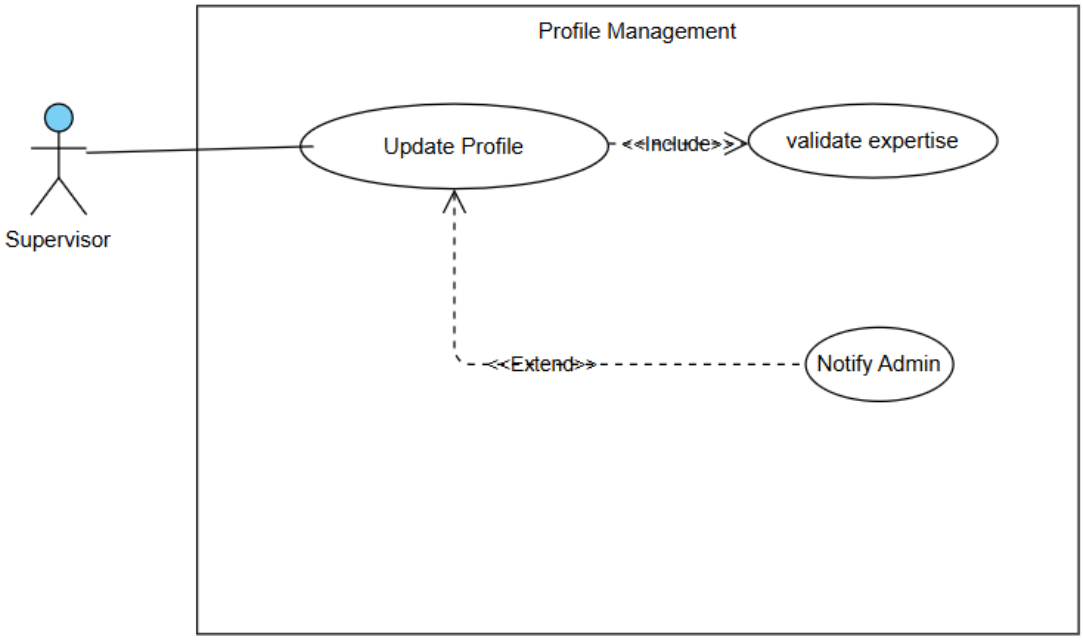


Fig 3.1.2.2. Profile management process for supervisor portal

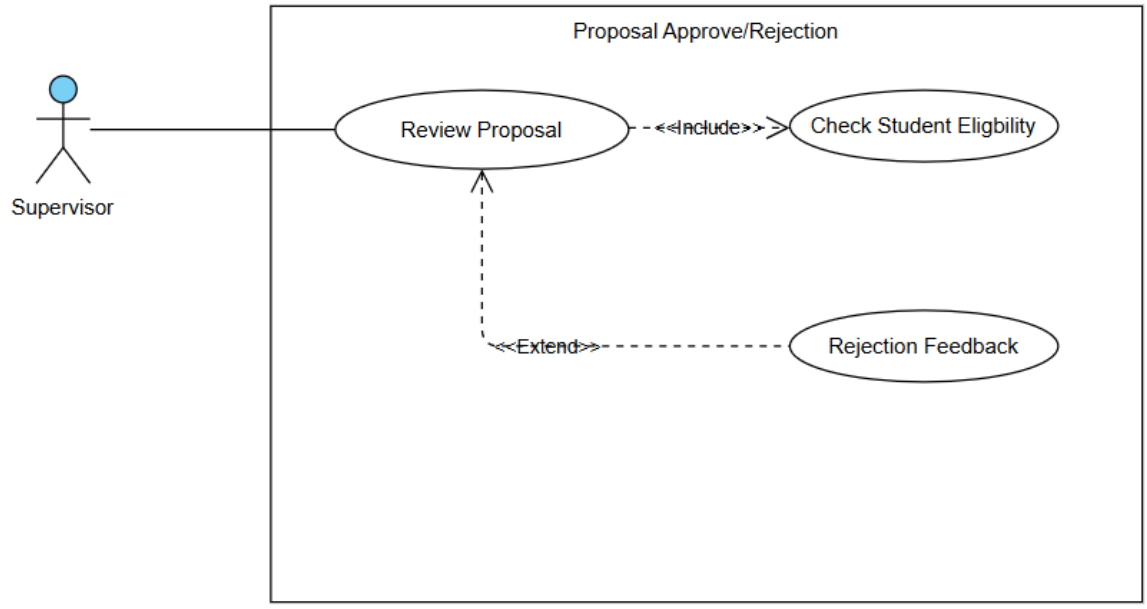


Fig 3.1.2.3. Proposal approval/rejection process for supervisor portal

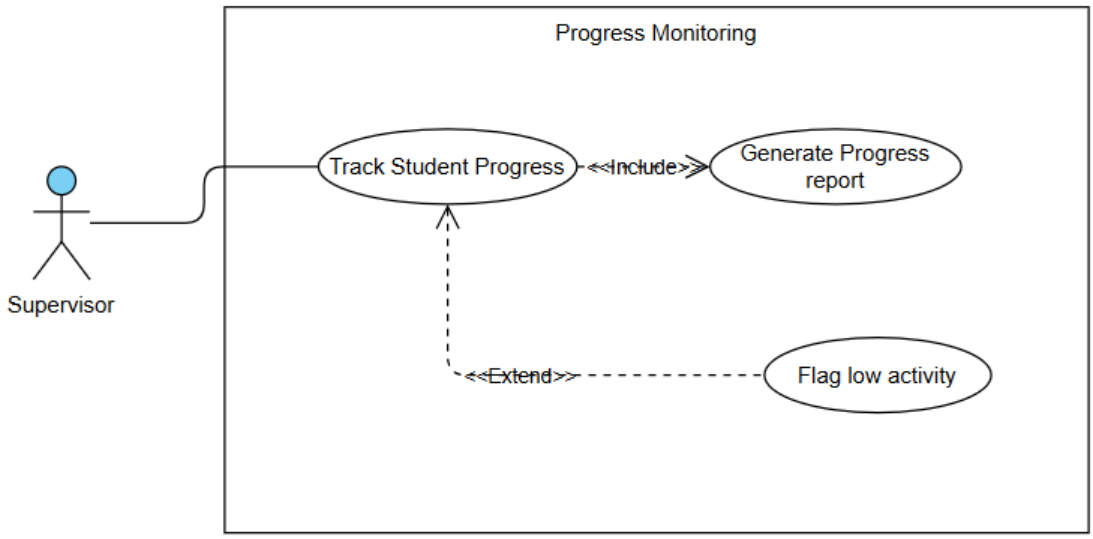


Fig 3.1.2.4. Progress monitoring process for supervisor portal

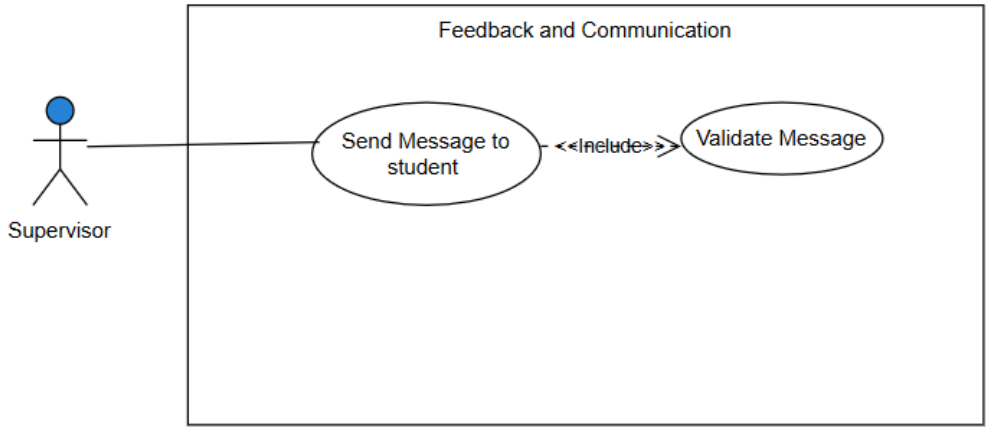


Fig 3.1.2.5. Feedback and communication process for supervisor portal

3.1.2.6 Explained Detail for Supervisor Portal Based System

ID	Use Case	Primary Actor	Secondary Actor	Includes (Mandatory)	Extends (Conditional)	Trigger	Pre-Conditions	Post-Conditions
UC-1	Authenticate via Gmail	Supervisor	System	Validate Credentials	Handle Login Failure	Supervisor accesses portal	Valid university email exists	Access granted or error shown
UC-2	Update Profile	Supervisor	Admin	Validate Expertise Input	Notify Admin (if availability <5hrs)	Supervisor edits profile	Supervisor is logged in	Profile updated; admin alerted if critical change
UC-3	Review Proposal	Supervisor	System, Student	Check Student Eligibility	Provide Rejection Feedback	New proposal assigned	Proposal status = "Pending"	Status updated to Approved/Rejected
UC-4	Track Student Progress	Supervisor	System, GitHub	Generate Progress Report	Flag Low Activity (no commits in 2wks)	Supervisor opens progress dashboard	Student has active project	Progress visualized; alerts triggered if inactive
UC-5	Send Message to Student	Supervisor	Student	Validate Message Content	Escalate Urgent Query (High Priority)	Supervisor initiates message	Student is assigned to supervisor	Message delivered; escalated if urgent

3.1.3 Use Case 3: Coordinator Portal

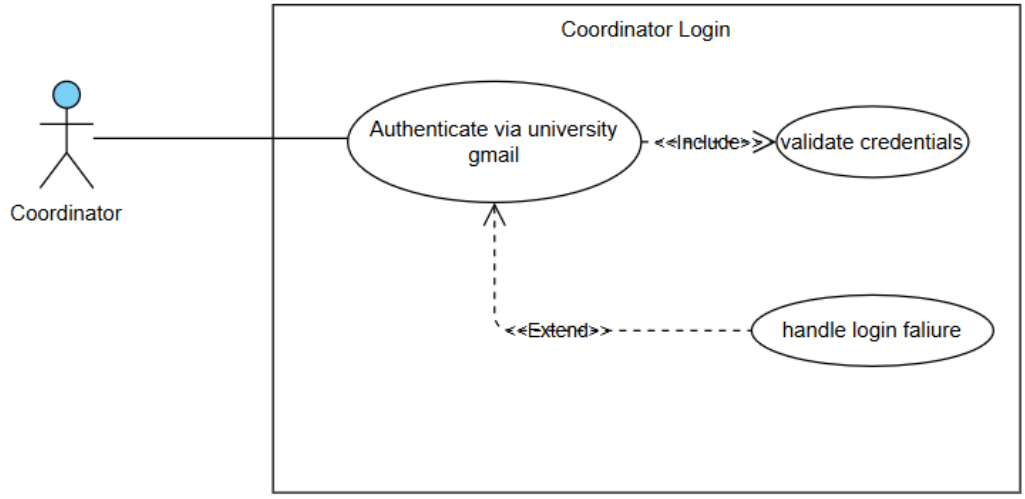


Fig 3.1.3.1. Login process for coordinator portal

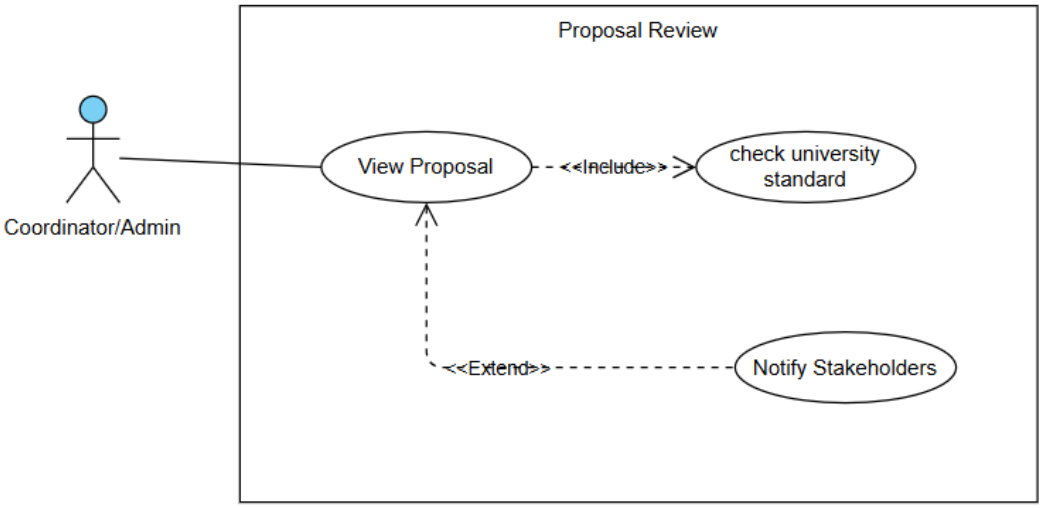


Fig 3.1.3.2. Proposal review process for coordinator portal

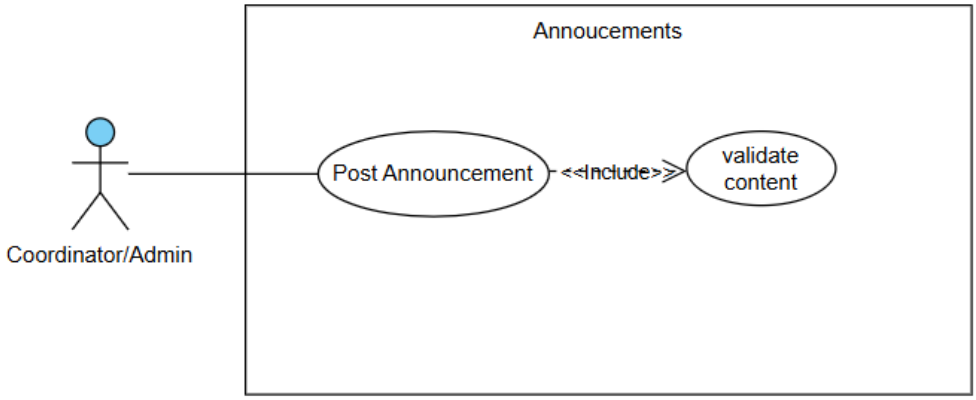


Fig 3.1.3.3. Announcement posting process for coordinator portal

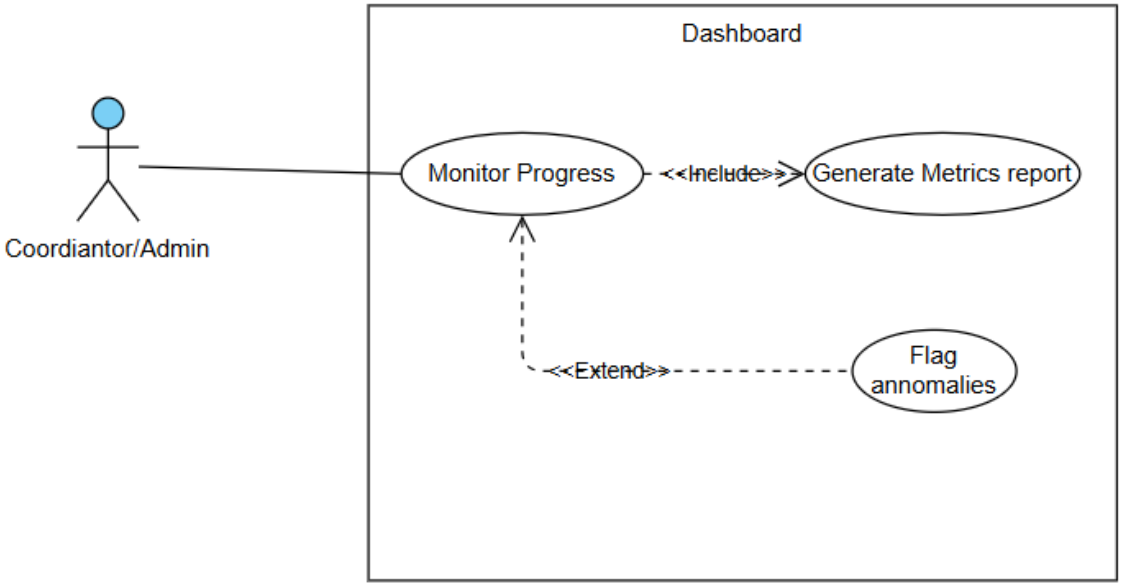


Fig 3.1.3.4. Dashboard visualization for coordinator portal

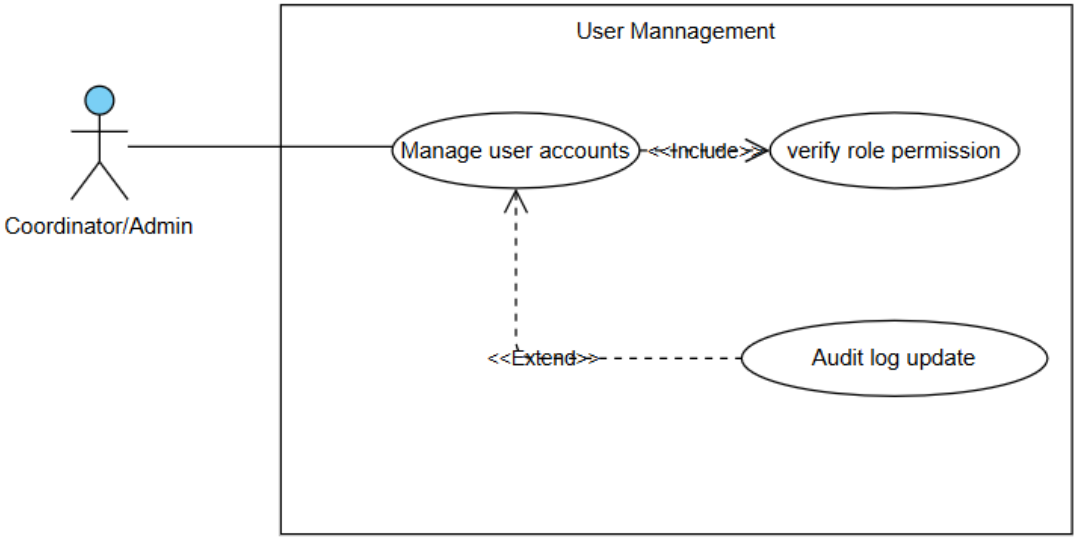


Fig 3.1.3.5. User management process for coordinator portal

3.1.3.2 Explained Detail for Coordinator Portal Based System

Use Case ID	Use Case Name	Actor	Description	Preconditions	Postconditions
UC1	Coordinator Login	Coordinator/Admin	Allows coordinator/admin to log in using university Gmail credentials.	Coordinator/Admin must have valid credentials.	User is authenticated and redirected to dashboard.
UC2	Proposal Review	Coordinator	Review, approve, or reject student project proposals. Notifications sent to students and supervisors.	Proposal is submitted by student.	Proposal status is updated, and notifications sent.
UC3	Post Announcements	Coordinator	Create and post announcements such as deadlines, updates, and guidelines.	Coordinator is logged in.	Announcement is visible to relevant users.
UC4	View Dashboard	Coordinator/Admin	Display key metrics: project progress, supervisor allocations, departmental stats.	Coordinator/Admin is logged in.	Dashboard with real-time data is displayed.
UC5	Manage Users	Admin	Add, remove, or update user accounts; assign roles and permissions.	Admin is logged in with proper privileges.	Users are updated in the system database.

Chapter 4:

Design

4.1 Architecture Diagram of System

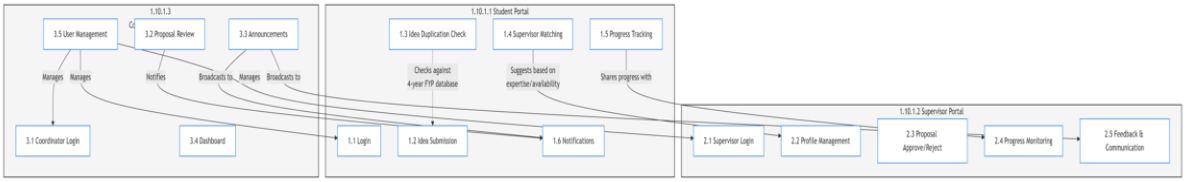


Figure 4.1. System architecture diagram

4.2 ERD Diagram of System

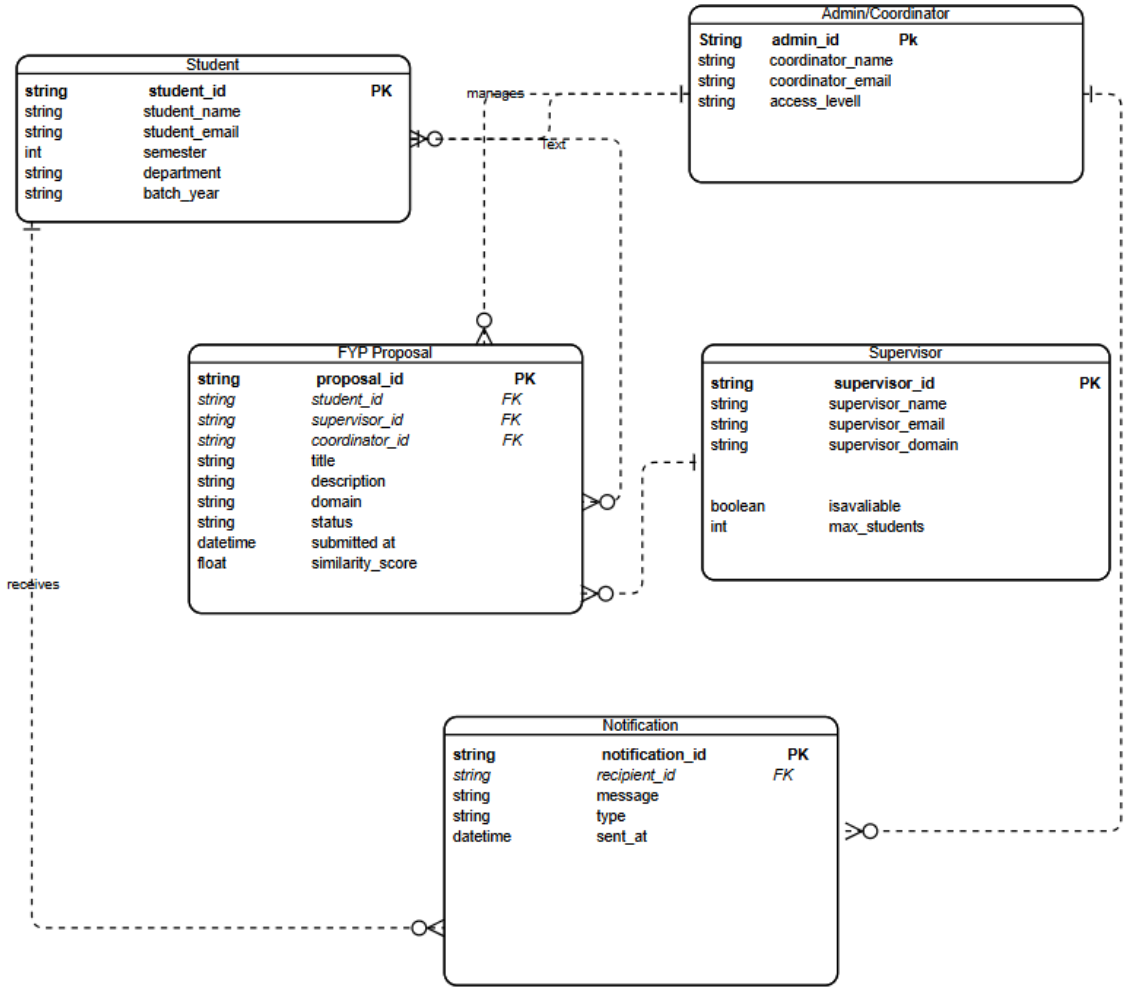


Figure 4.2. Entity-relationship diagram (ERD) of FYPAS

4.3 DFD Diagram Level 1

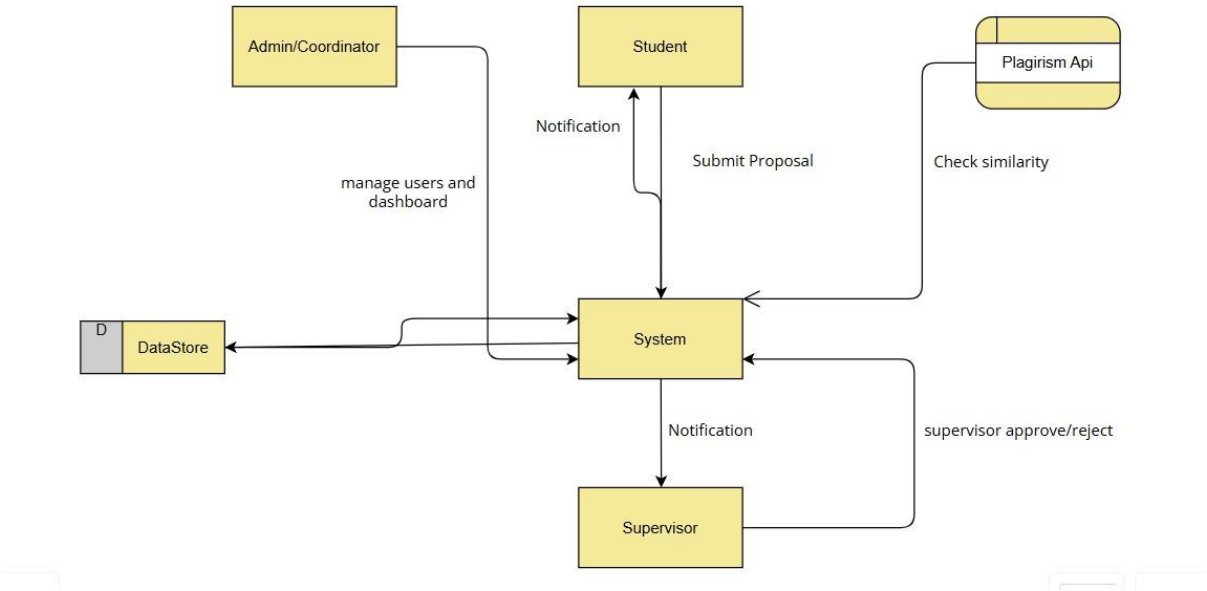


Figure 4.3. Data flow diagram (DFD) level 1

4.4 DFD Diagram Level 2

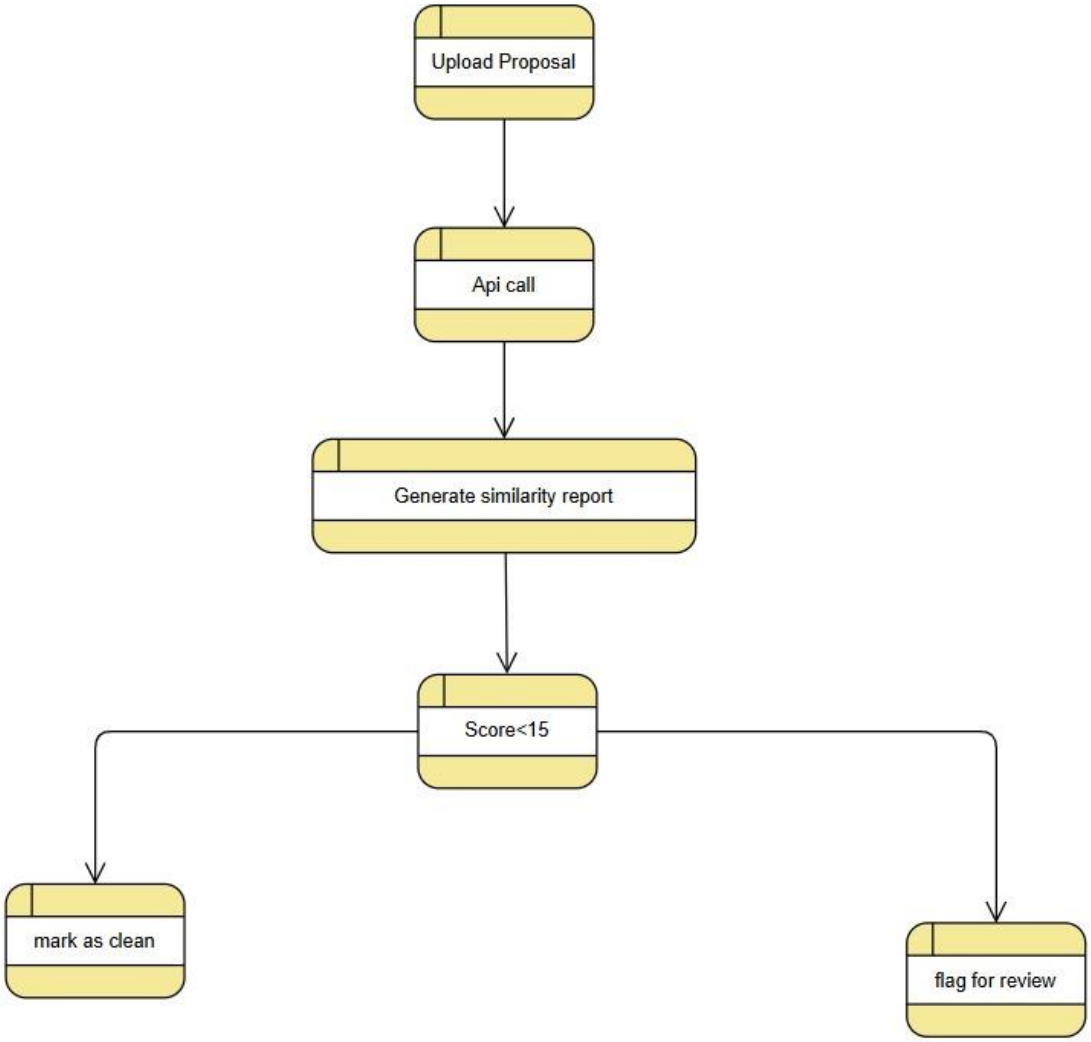


Figure 4.4. Data flow diagram (DFD) level 2

4.5 Class Diagram of System

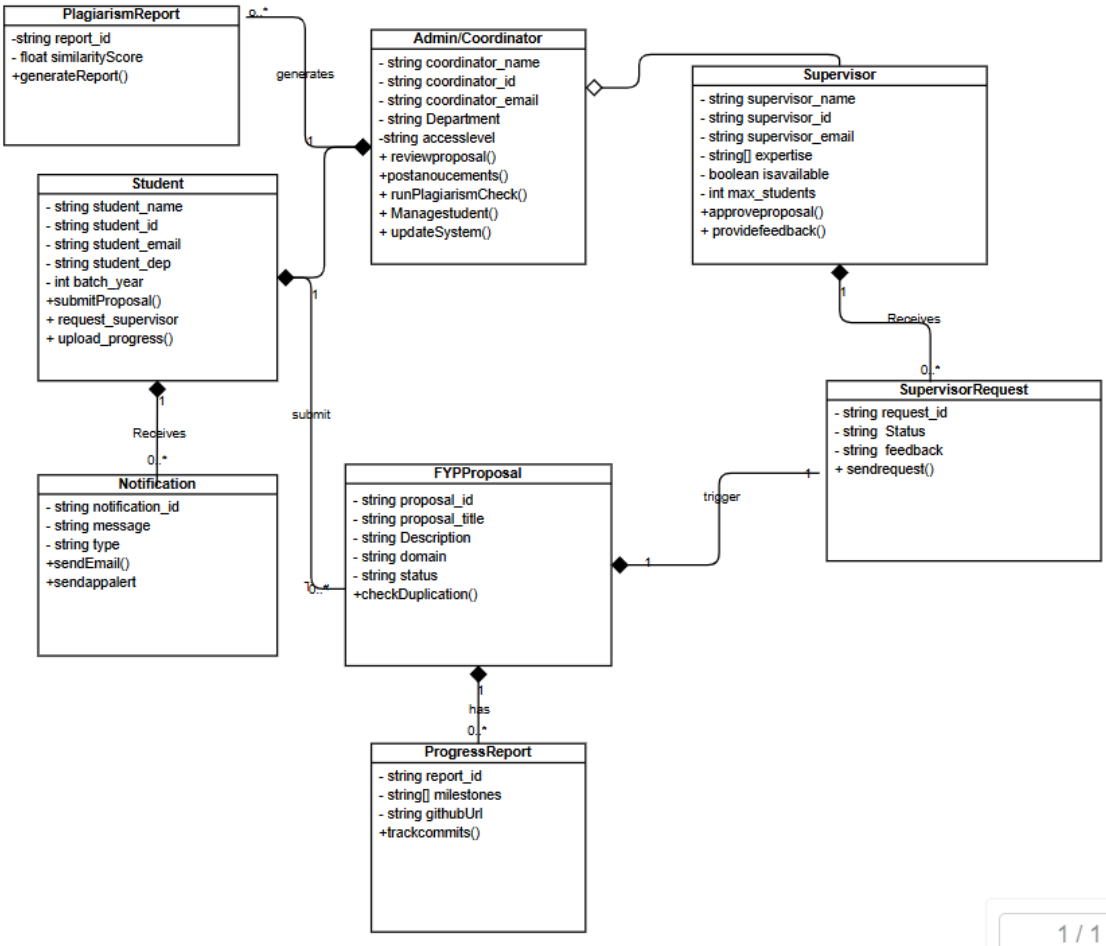


Figure 4.5. Class diagram of FYPAS

4.6 Activity Diagram:

4.6.1 Student Portal:

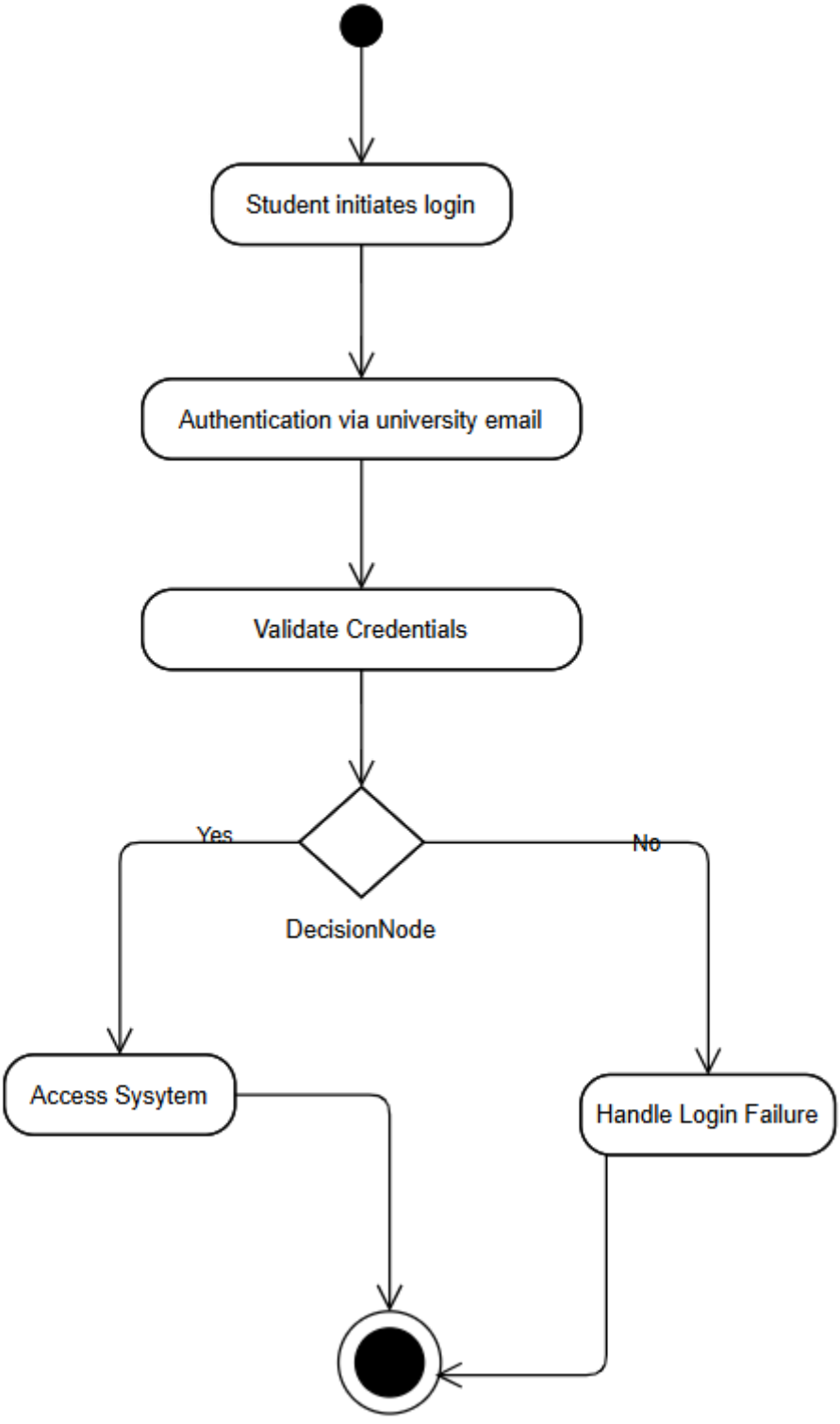


Fig 4.6.1.1. Login activity diagram for student portal

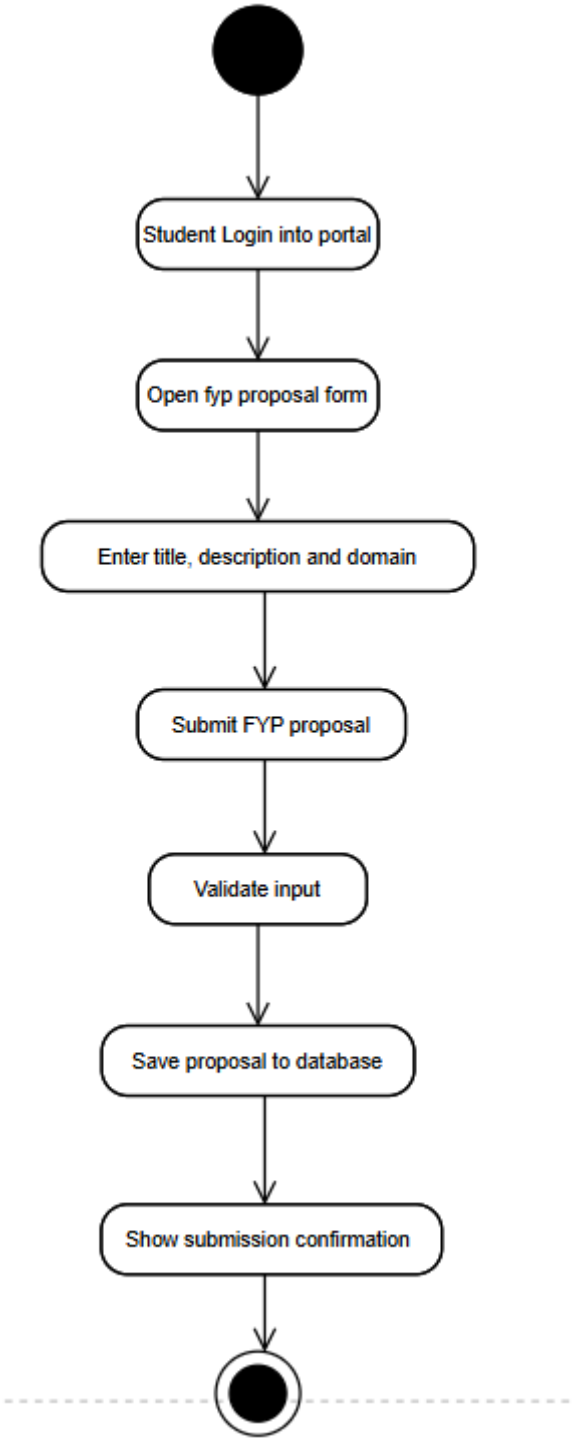


Fig 4.6.1.2. Idea submission activity diagram for student portal

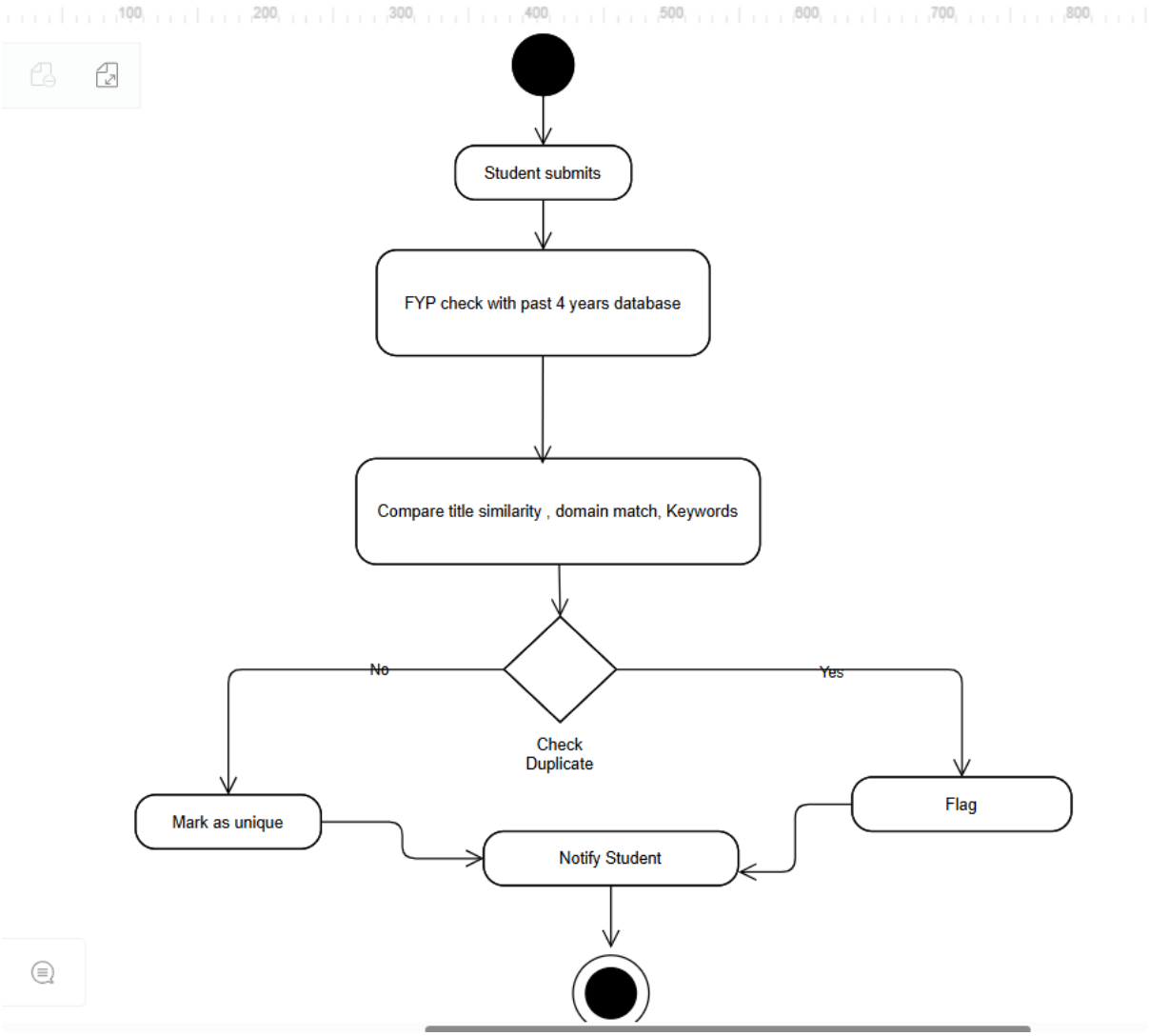


Fig 4.6.1.3. Idea duplication check activity diagram for student portal

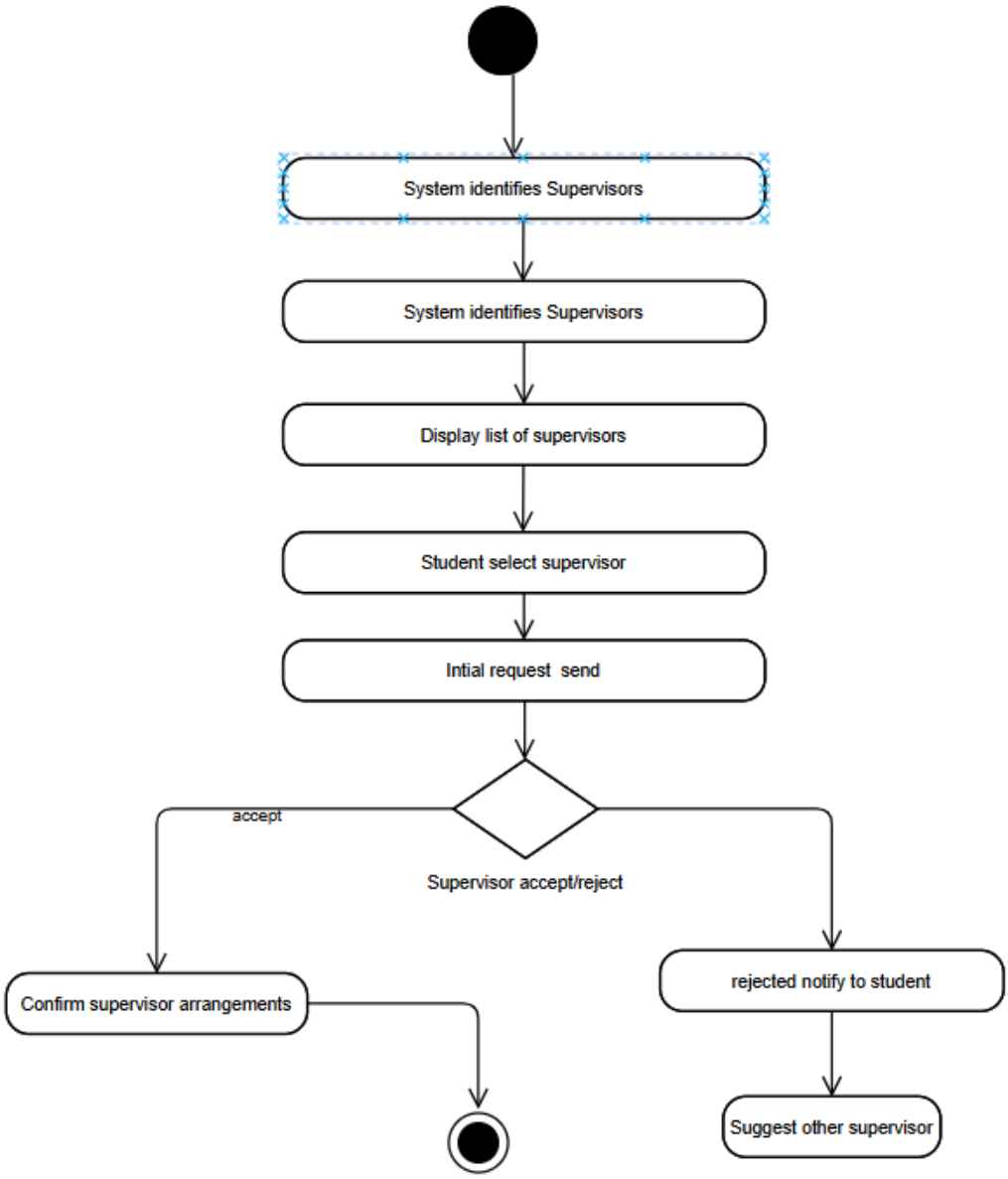


Fig 4.6.1.4. Supervisor matching activity diagram for student portal

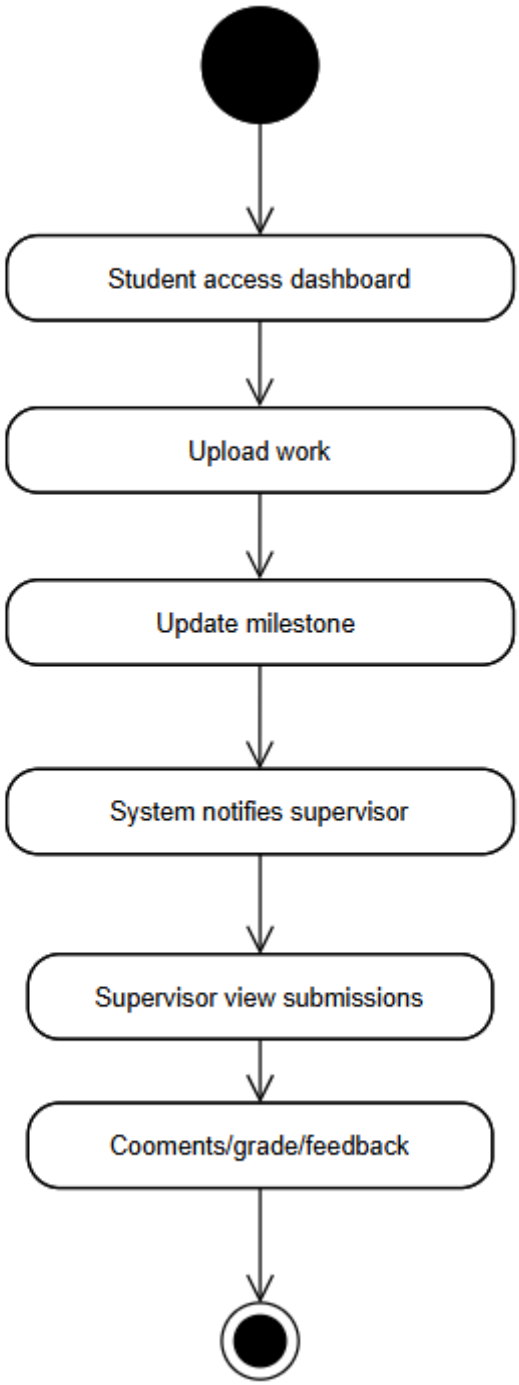


Fig 4.6.1.5. Progress tracking activity diagram for student portal

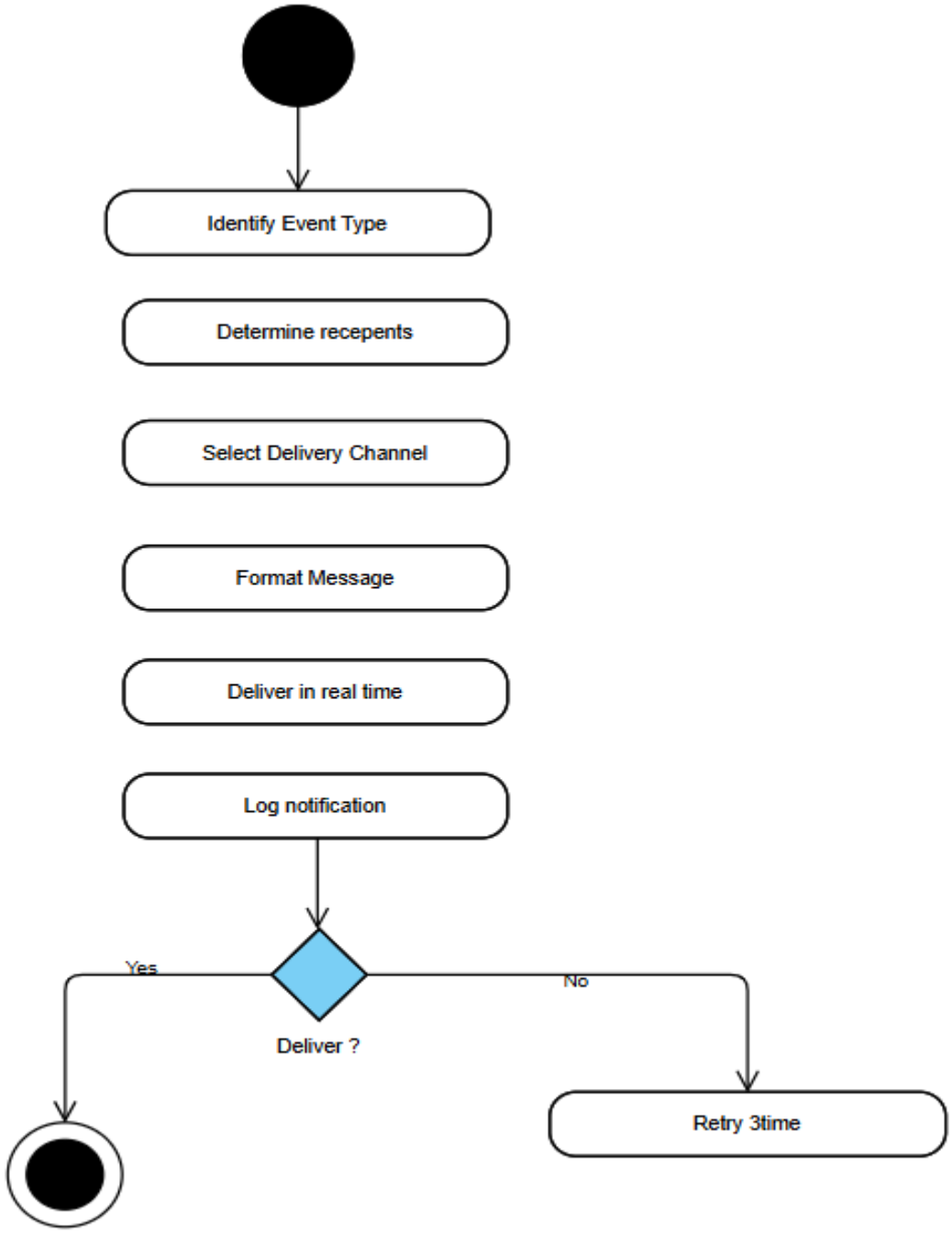


Fig 4.6.1.6. Notification activity diagram for student portal

4.6.2 Supervisor Portal:

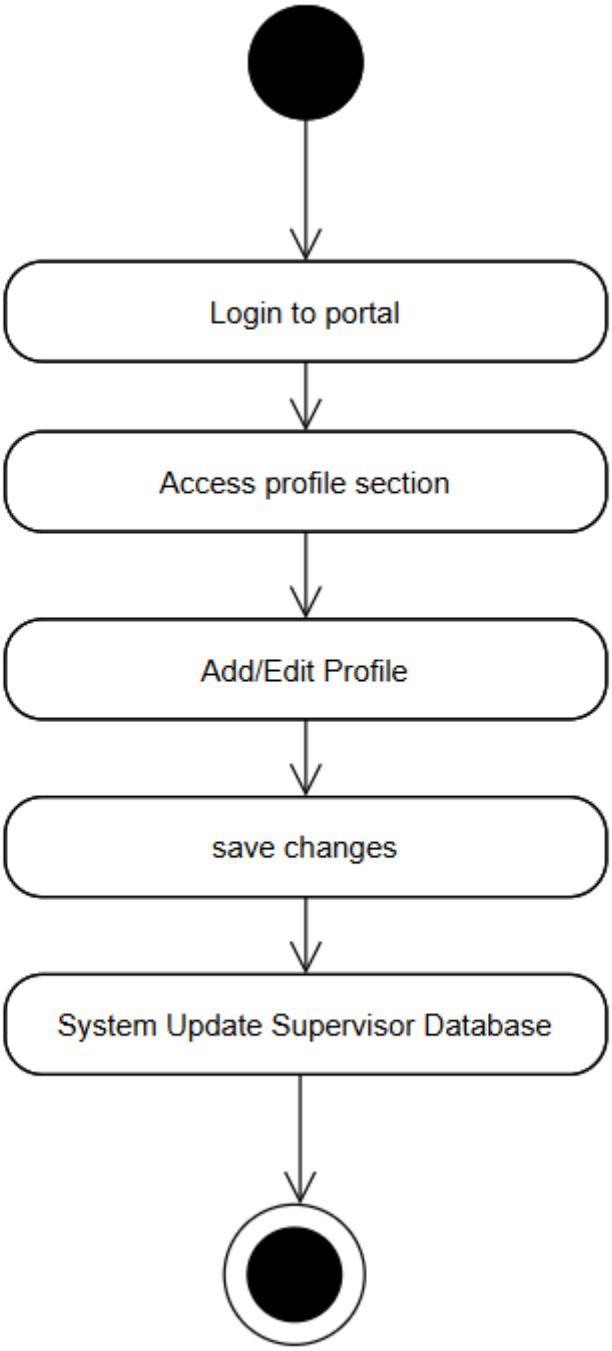


Fig 4.6.2.1. Profile management activity diagram for supervisor portal

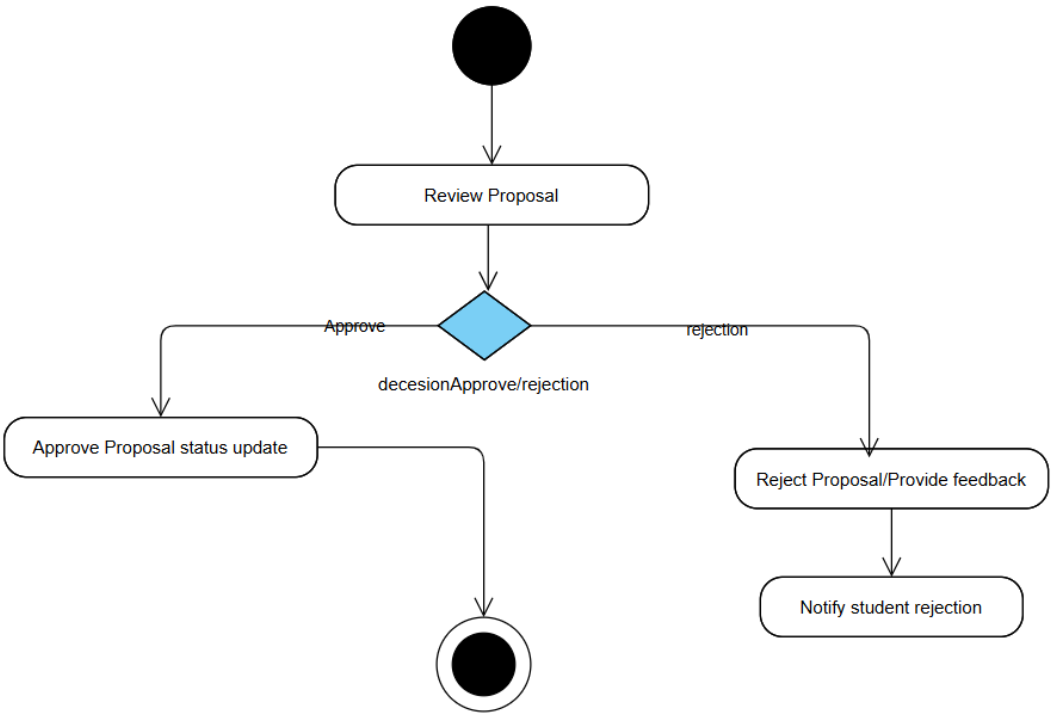


Fig 4.6.2.2. Proposal approval/rejection activity diagram for supervisor portal

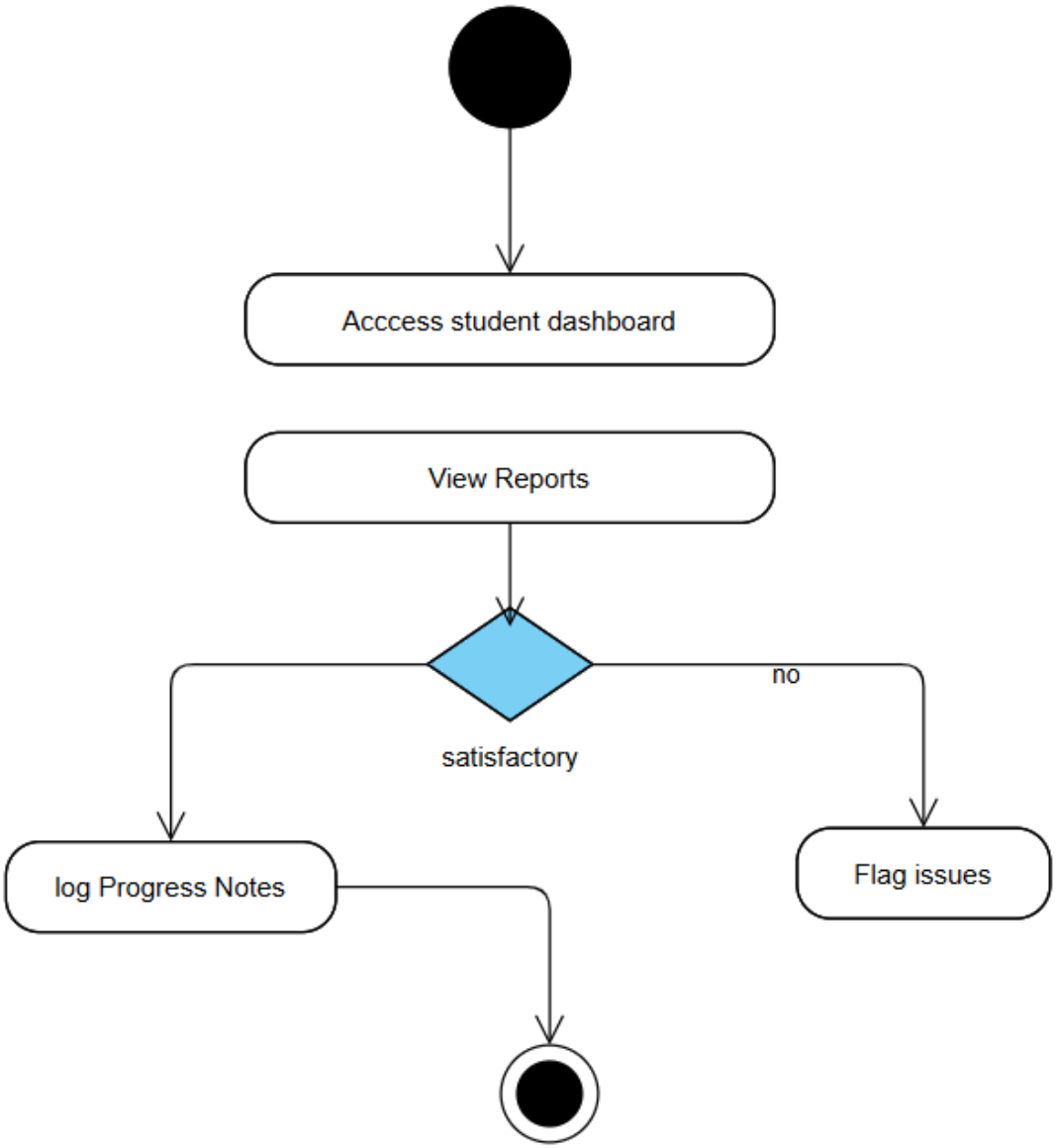


Fig 4.6.2.3. Progress monitoring activity diagram for supervisor portal

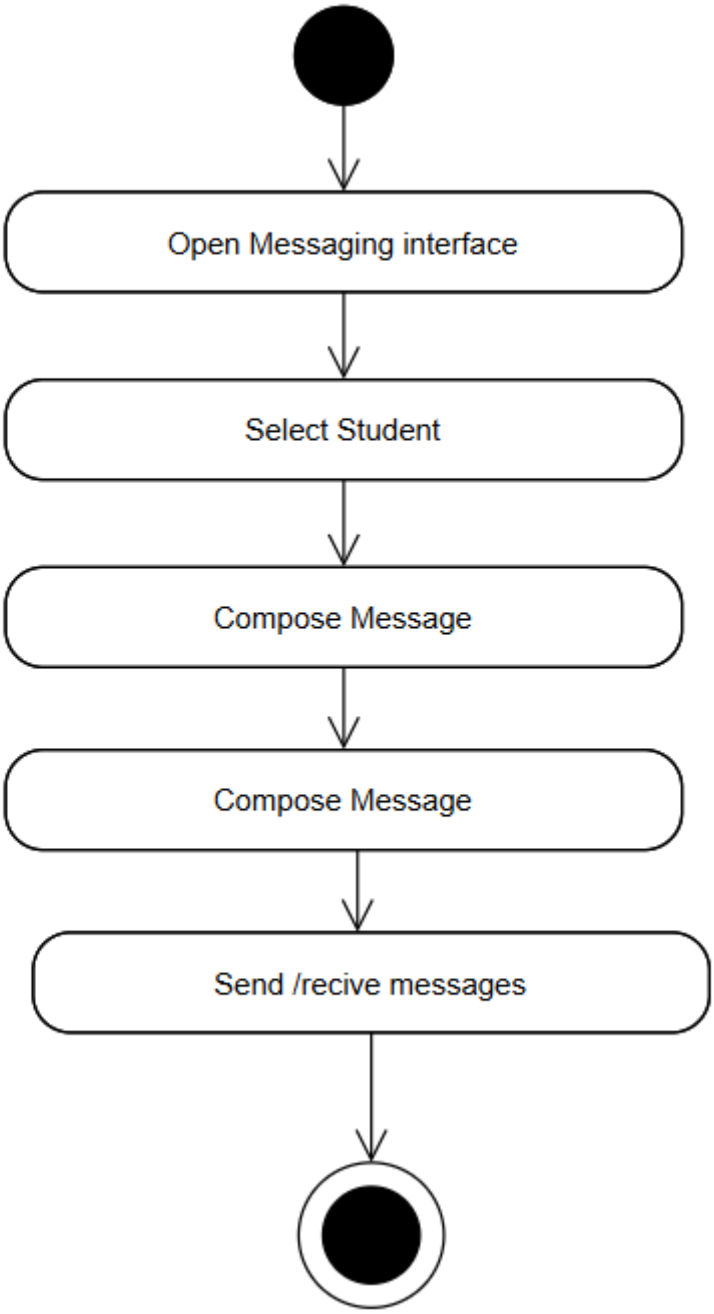


Fig 4.6.2.4. Feedback and communication activity diagram for supervisor portal

4.6.3 Coordinator Portal:

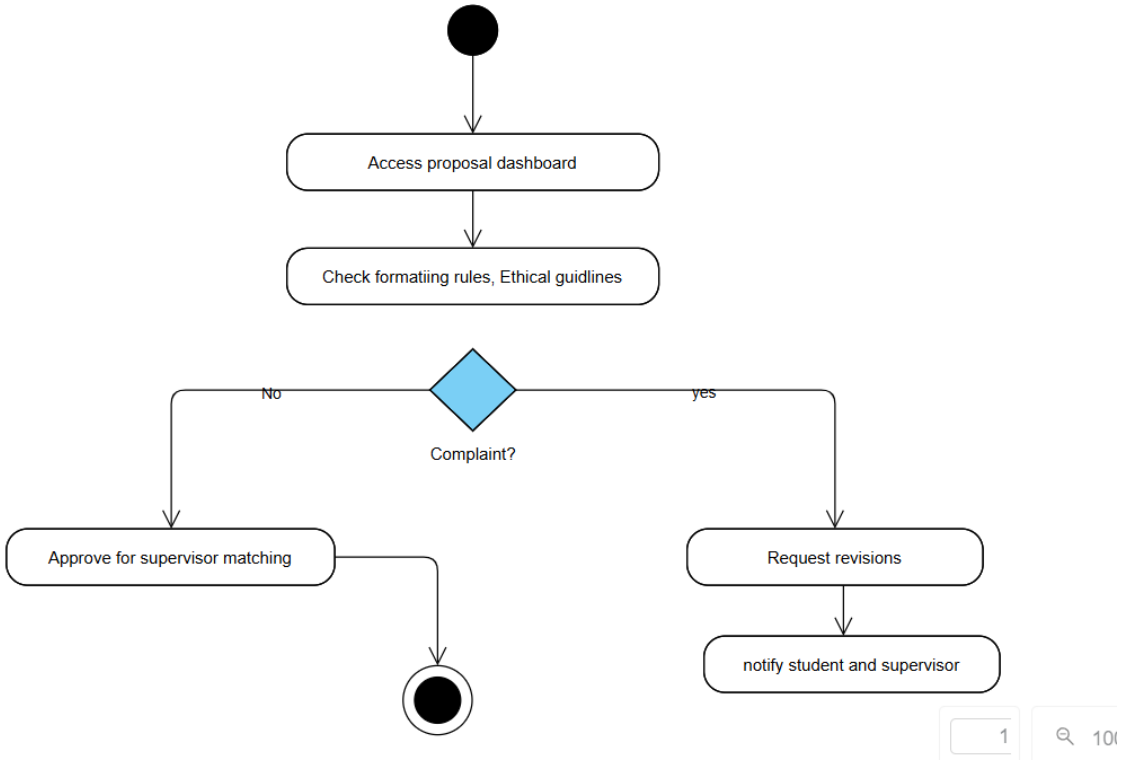


Fig 4.6.3.1. Feedback and communication activity diagram for coordinator portal

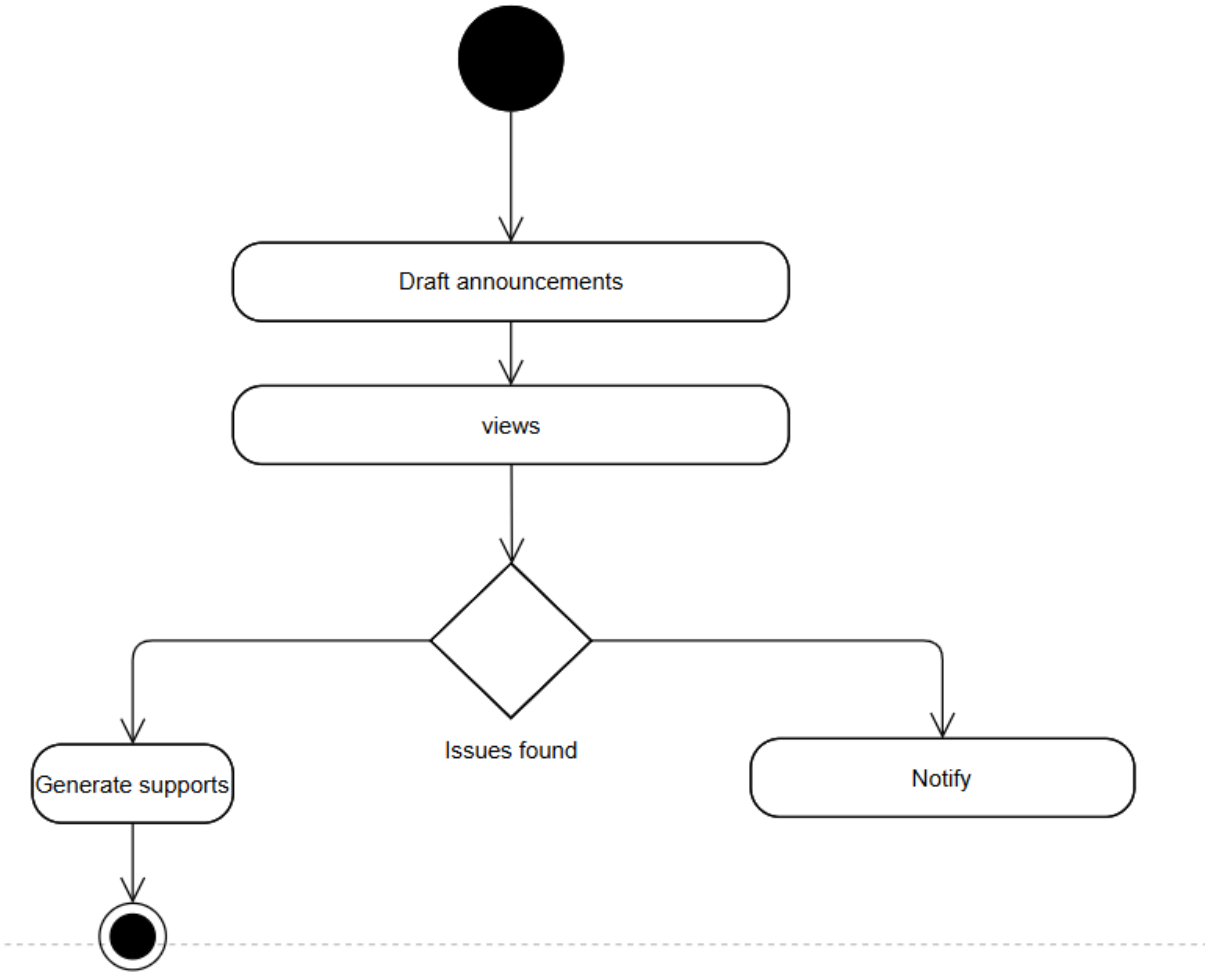


Fig 4.6.3.2. Post announcement activity diagram for coordinator portal

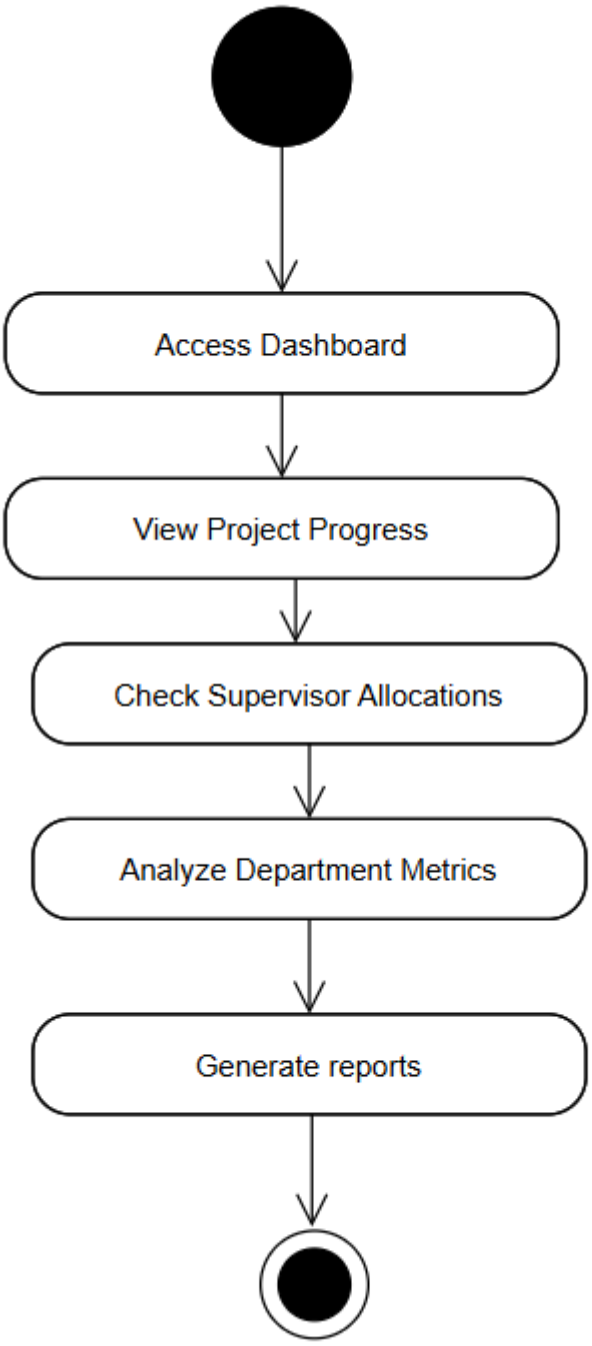


Fig 4.6.3.3. Dashboard activity diagram for coordinator portal

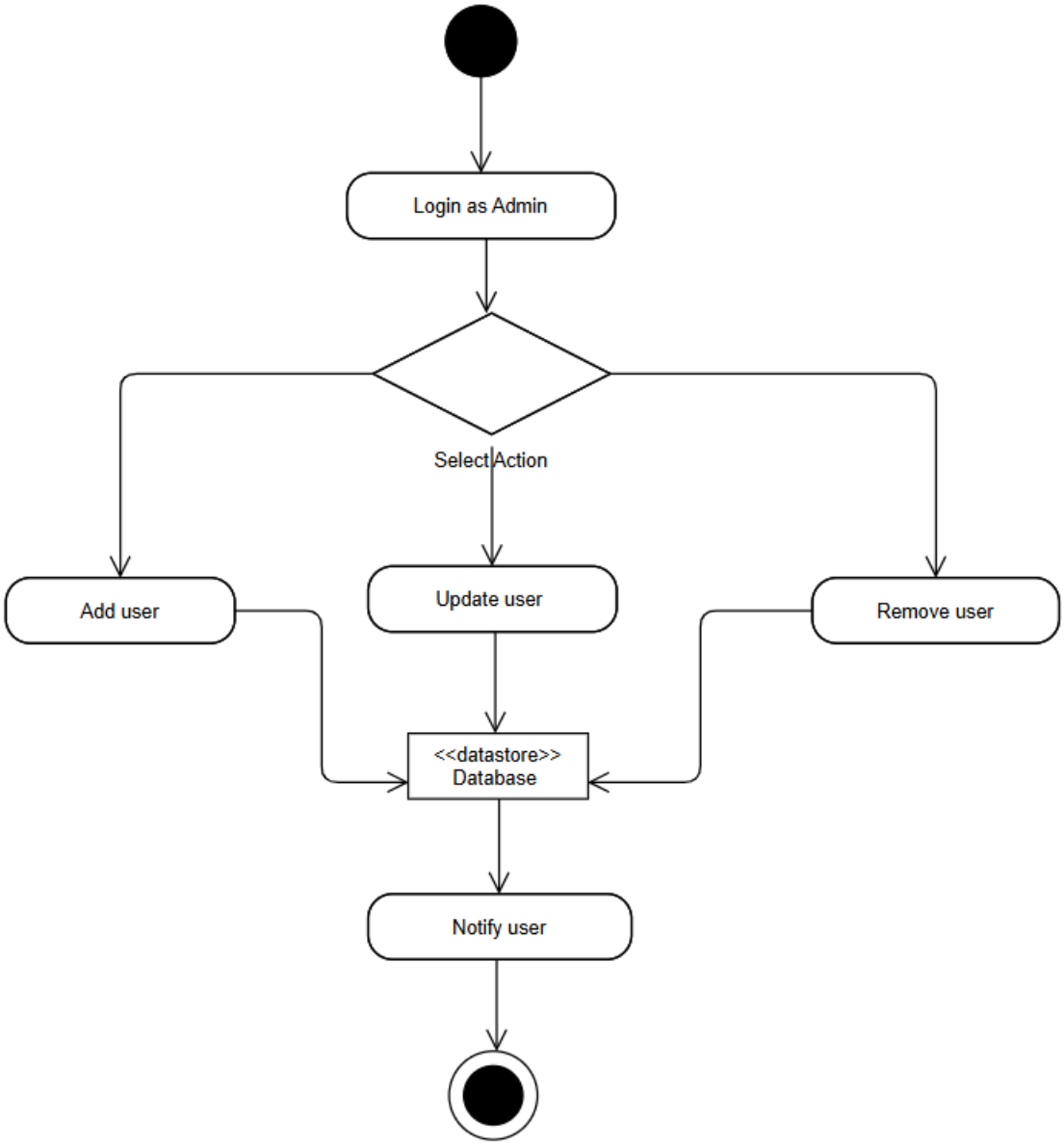


Fig 4.6.3.4. User management activity diagram for coordinator portal

4.7 Sequence Diagram

4.7.1 Student Portal

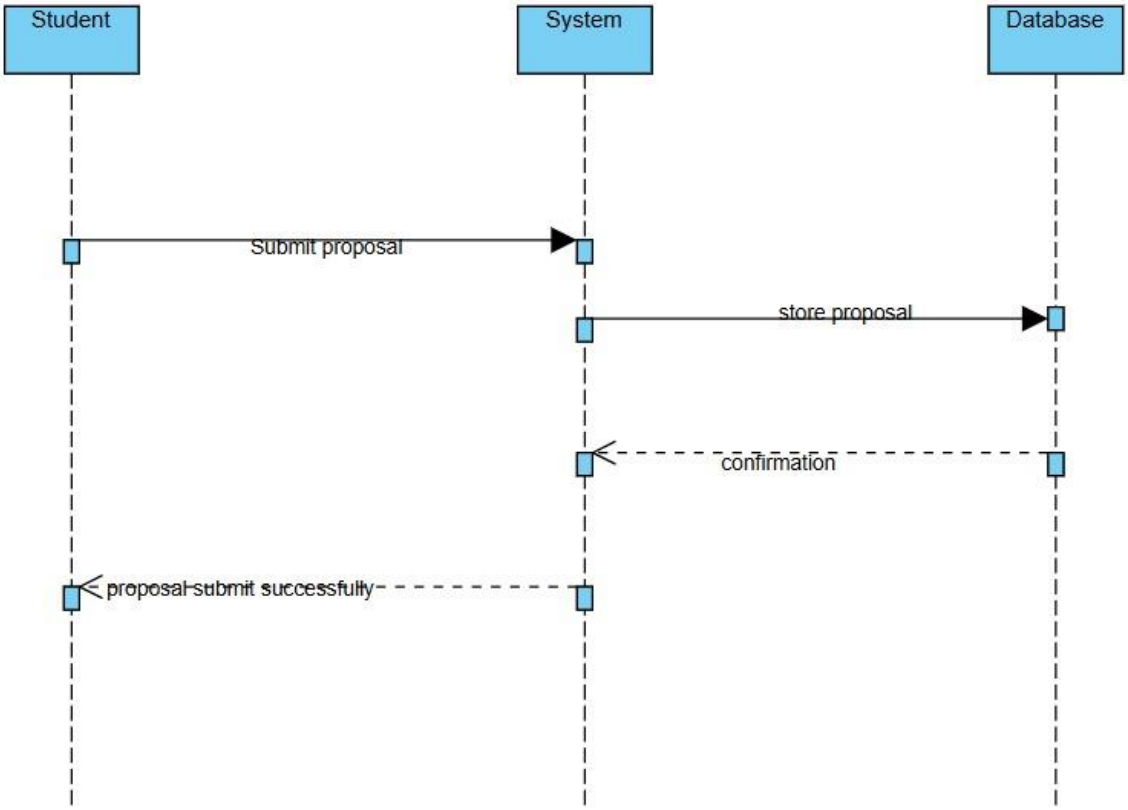


Fig 4.7.1.1. Idea submission sequence diagram for student portal

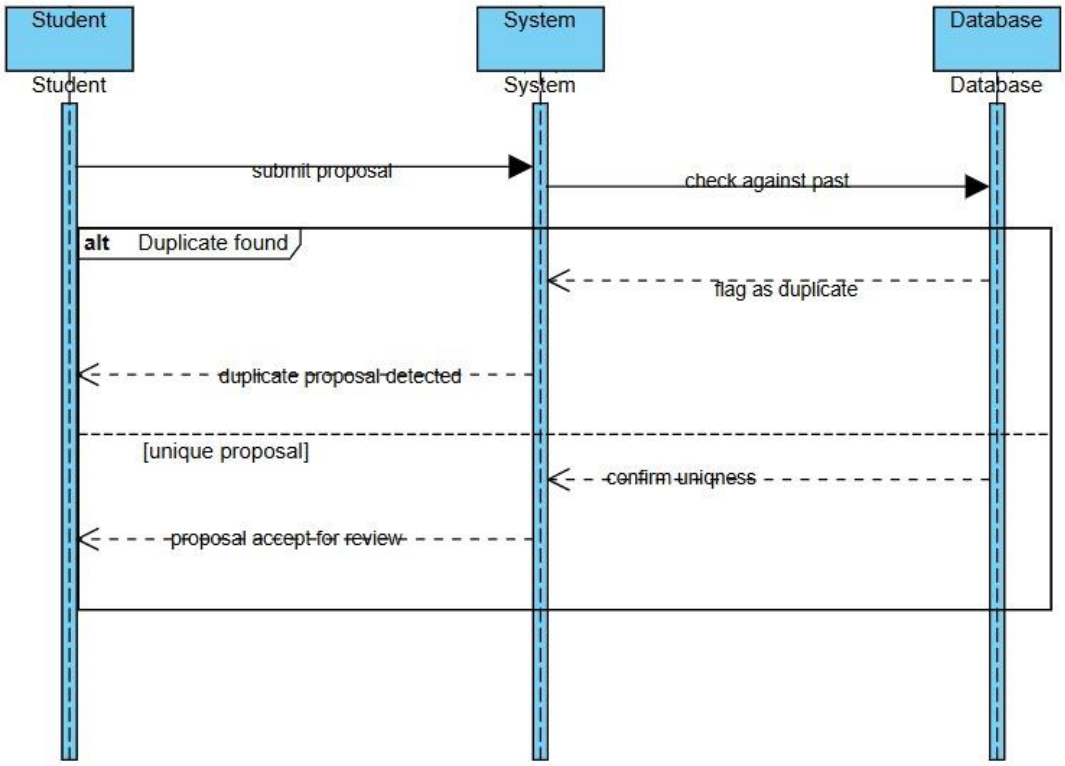


Fig 4.7.1.2. Idea duplication check sequence diagram for student portal

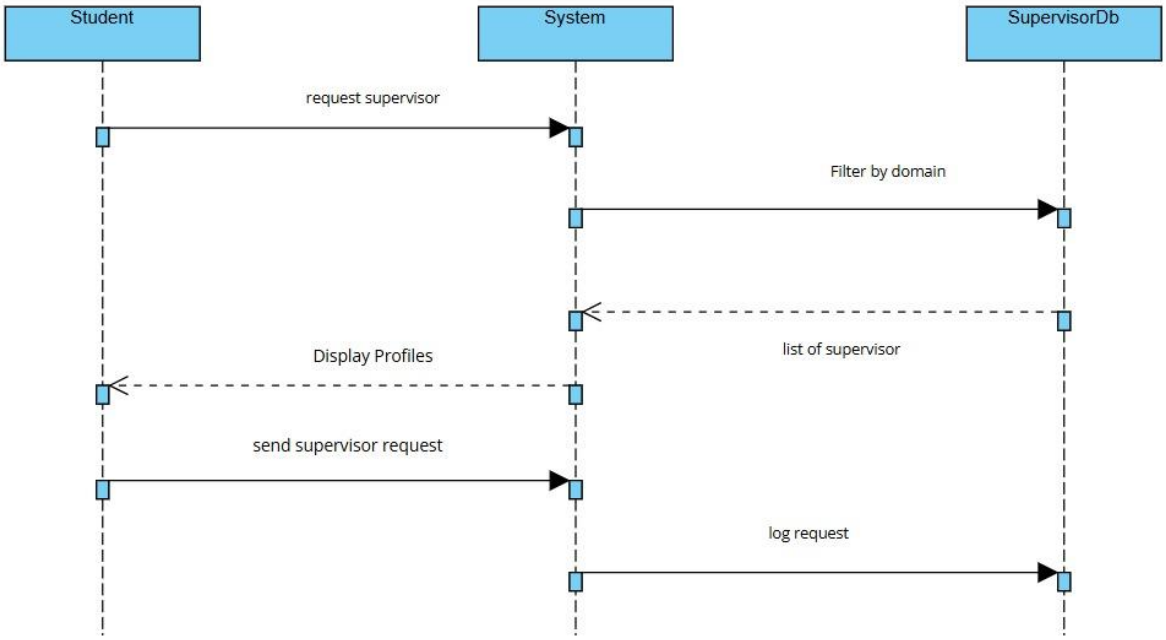


Fig 4.7.1.3. Supervisor matching sequence diagram for student portal

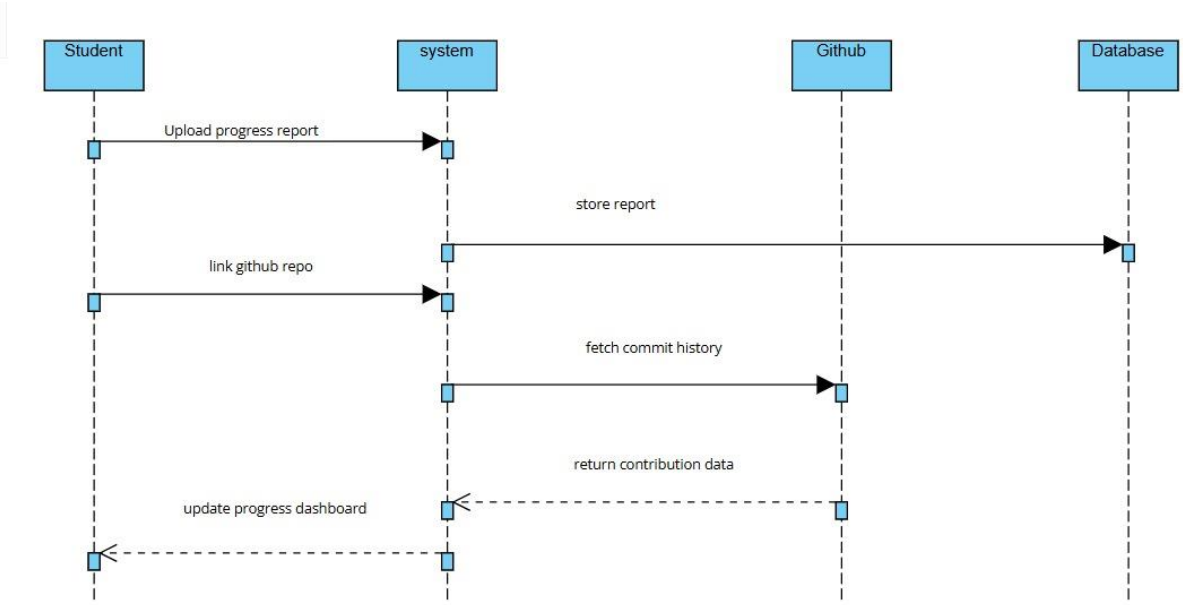


Fig 4.7.1.4. Progress tracking sequence diagram for student portal

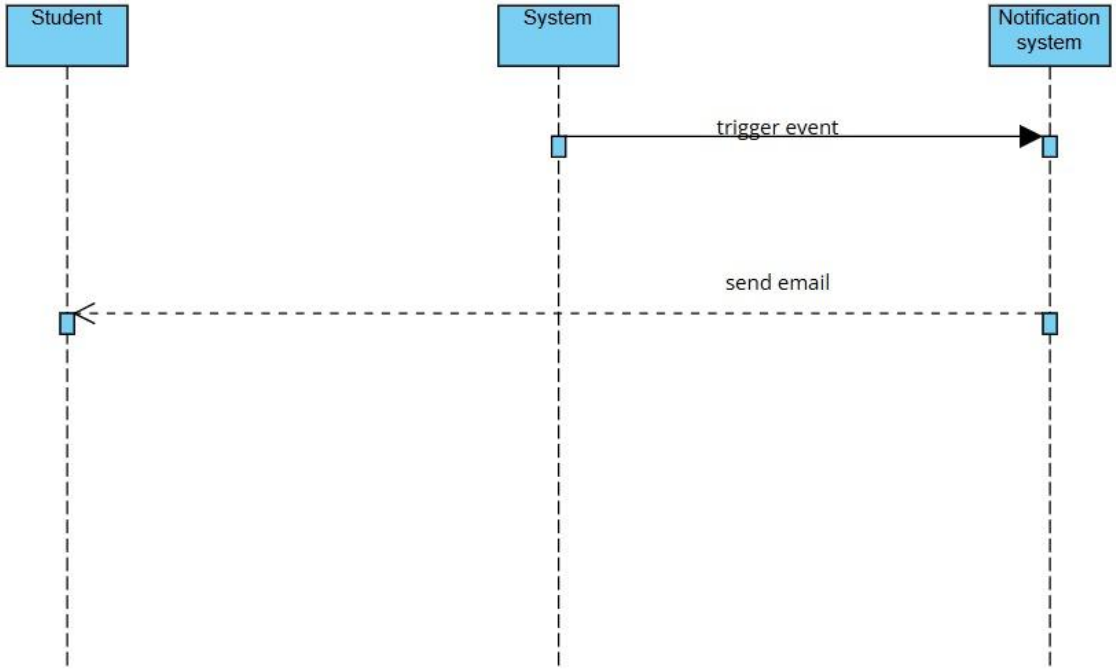


Fig 4.7.1.5. Notification sequence diagram for student portal

4.7.2 Supervisor Portal:

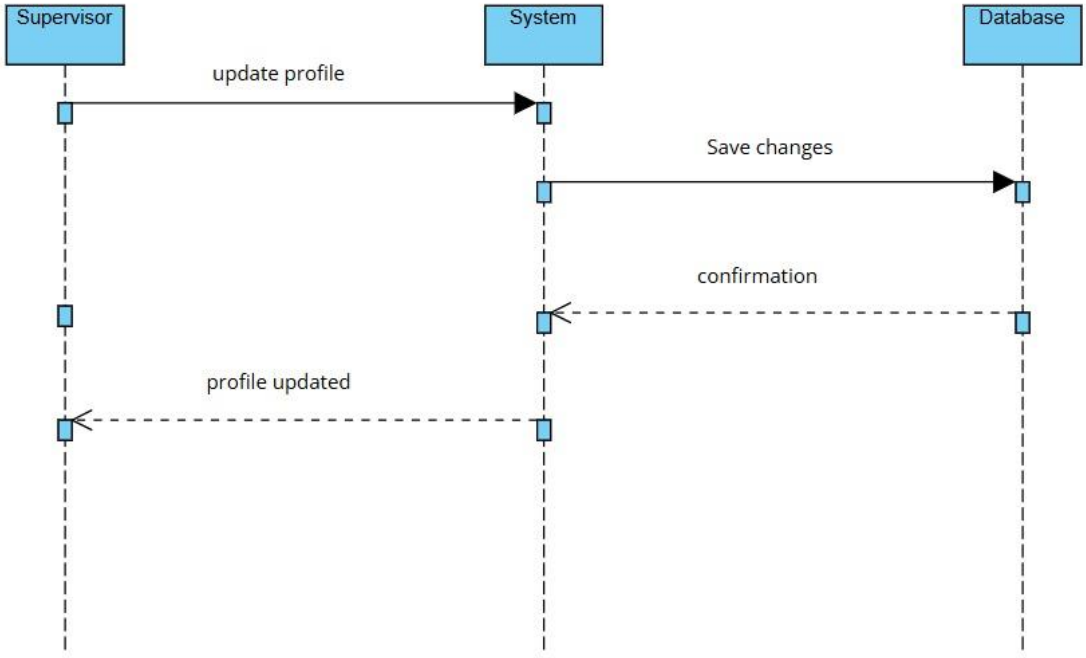


Fig 4.7.2.1. Profile management sequence diagram for supervisor portal

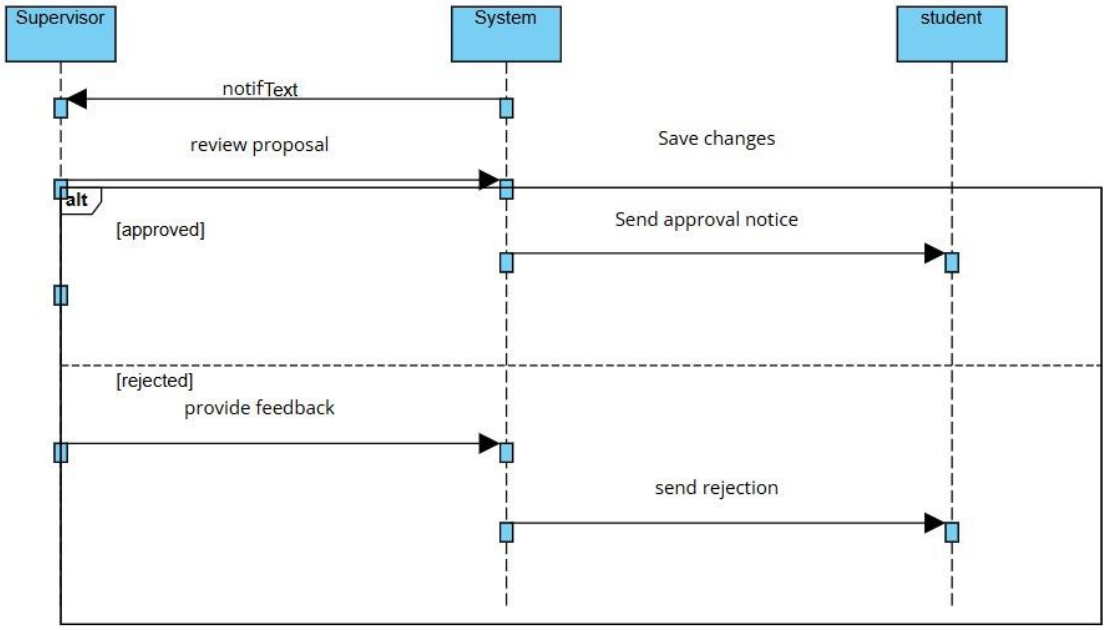


Fig 4.7.2.2. Proposal approval/rejection sequence diagram for supervisor portal

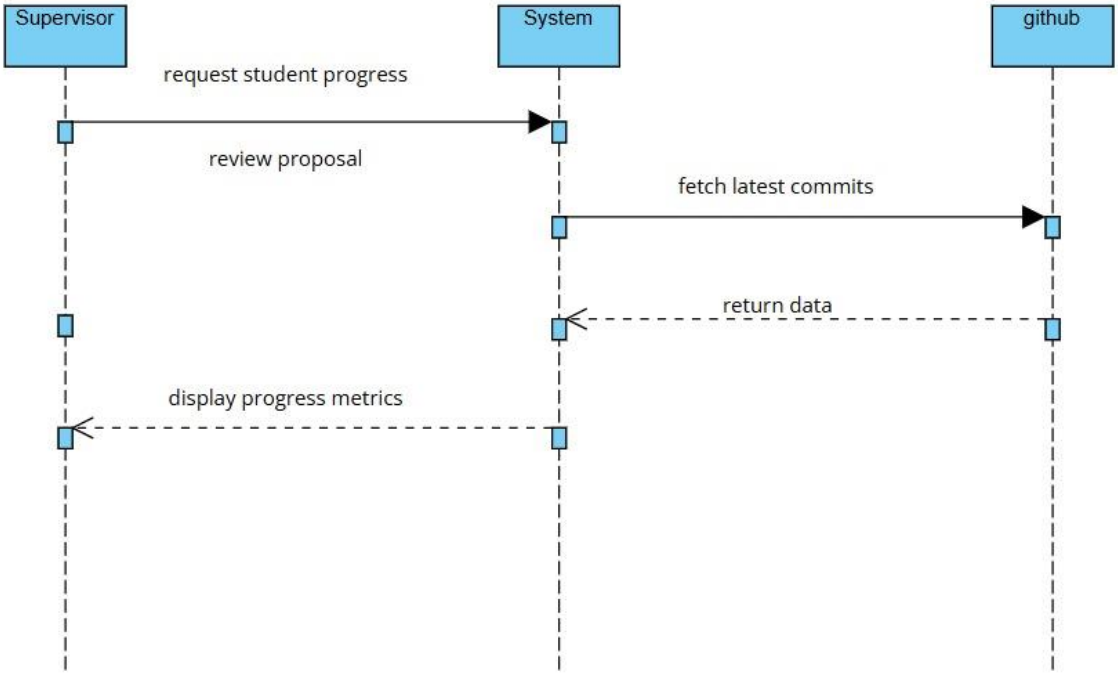


Fig 4.7.2.3. Progress monitoring sequence diagram for supervisor portal

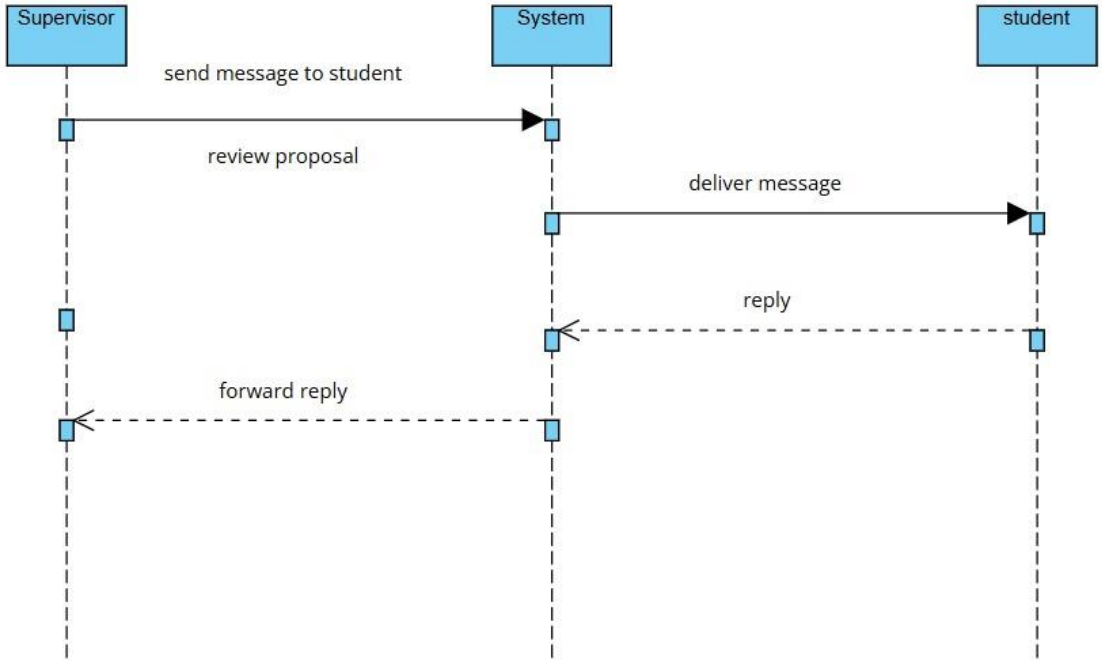


Fig 4.7.2.4. Feedback and communication sequence diagram for supervisor portal

4.7.3 Coordinator Portal:

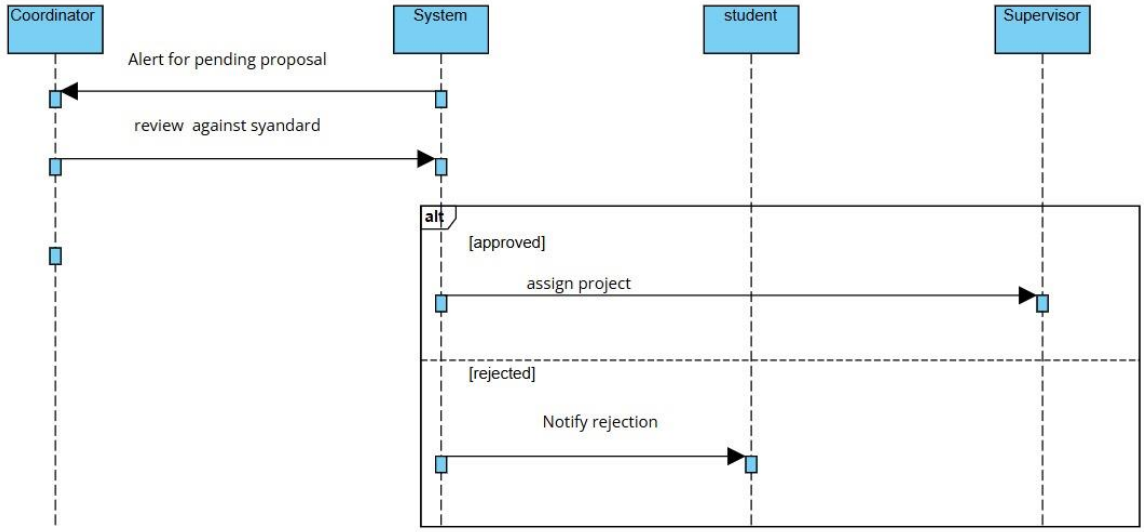


Fig 4.7.3.1. Proposal review sequence diagram for coordinator portal

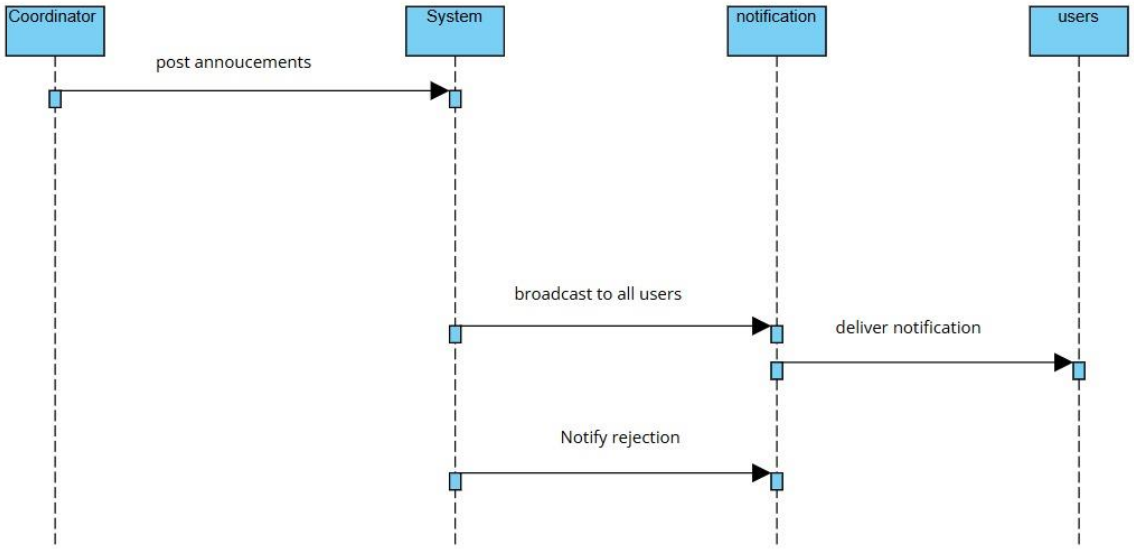


Fig 4.7.3.2. Announcements sequence diagram for coordinator portal

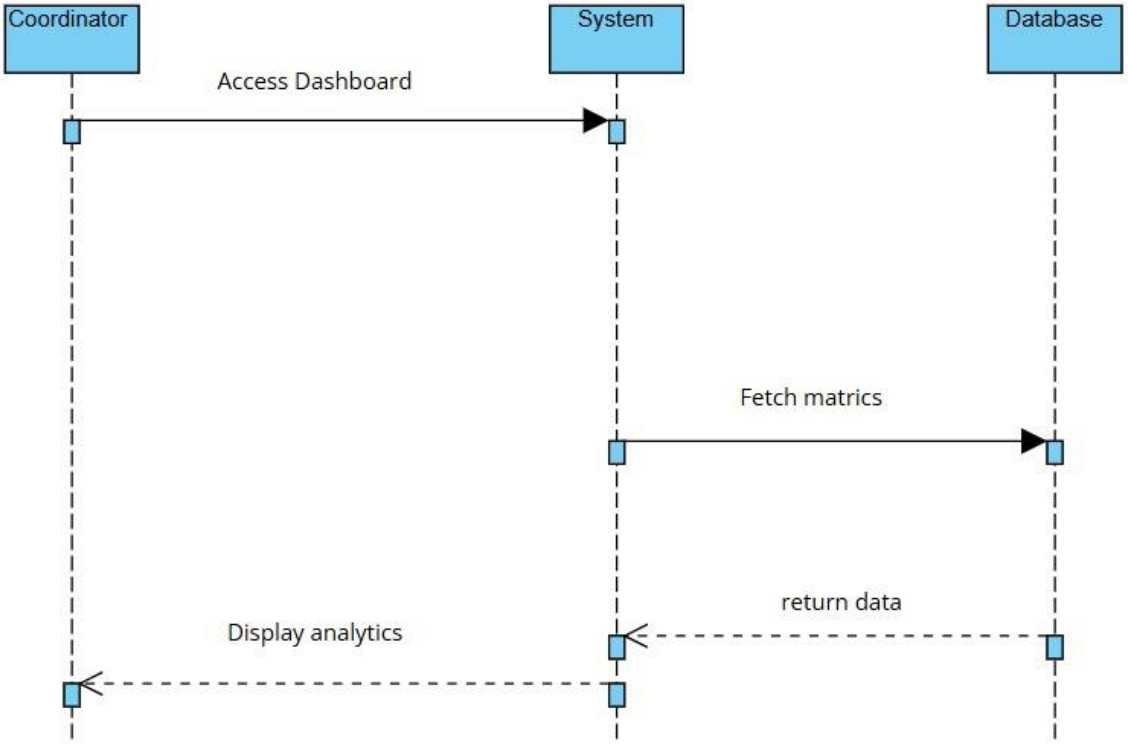


Fig 4.7.3.3. Dashboard sequence diagram for coordinator portal

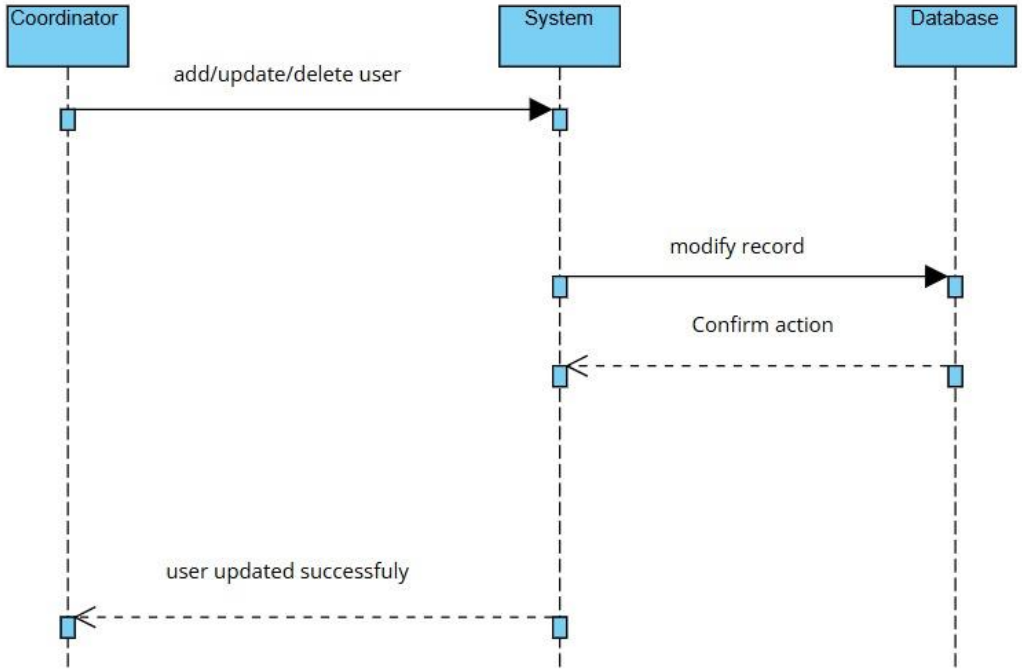


Fig 4.7.3.4. User management sequence diagram for coordinator portal

4.8 Collaboration Diagram

4.8.1 Student Portal:

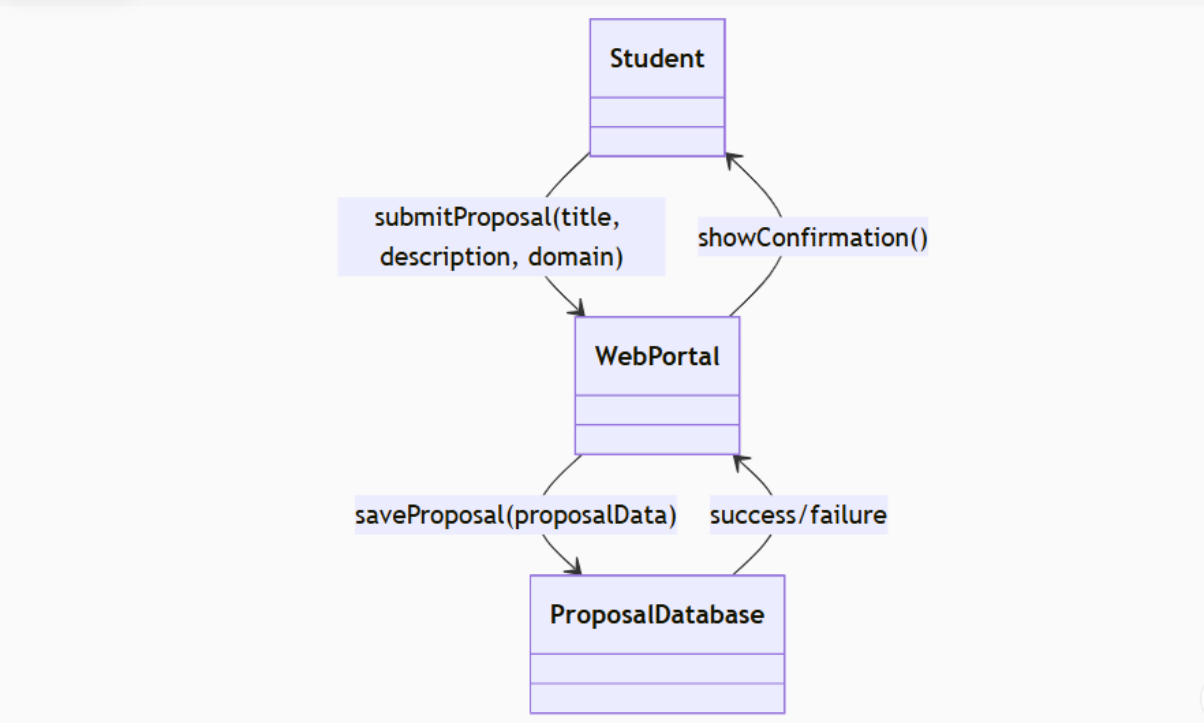


Fig 4.8.1.1. Idea submission collaboration diagram for student portal

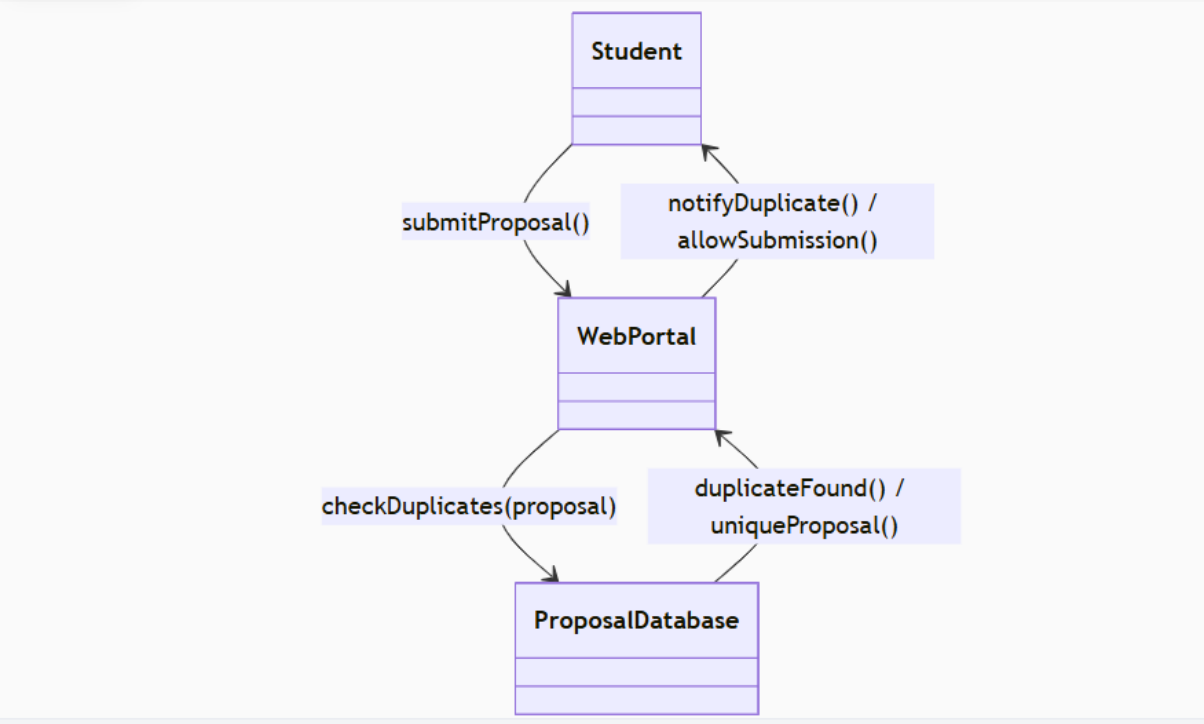


Fig 4.8.1.2. Idea duplication check collaboration diagram for student portal

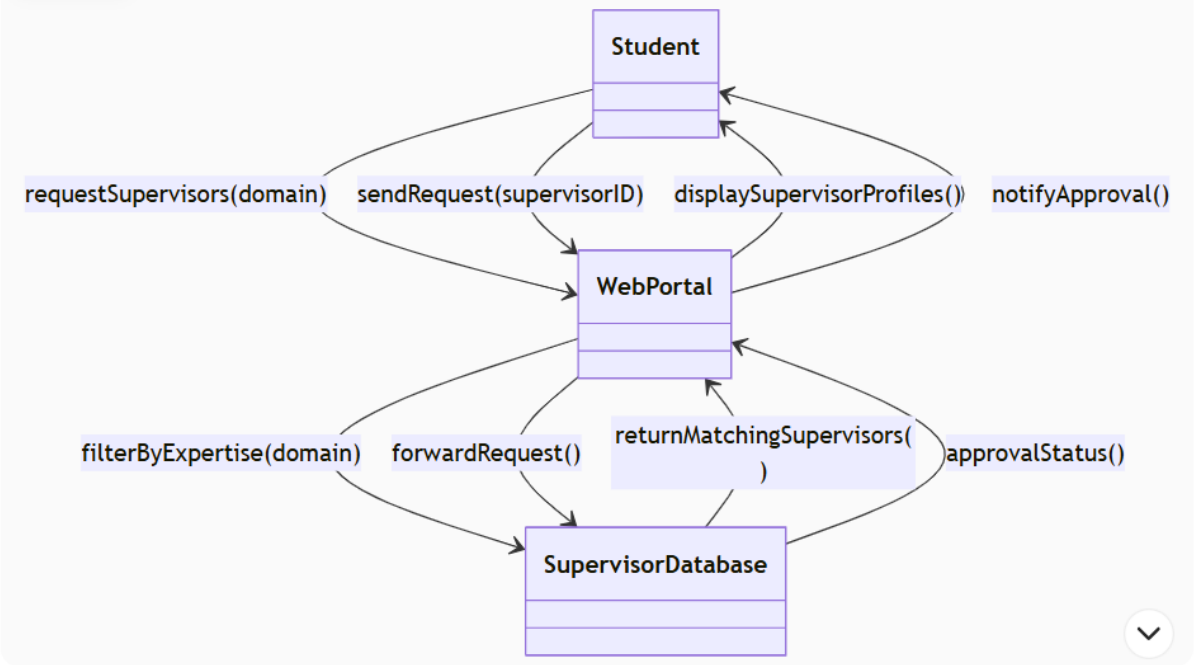


Fig 4.8.1.3. Supervisor matching collaboration diagram for student portal

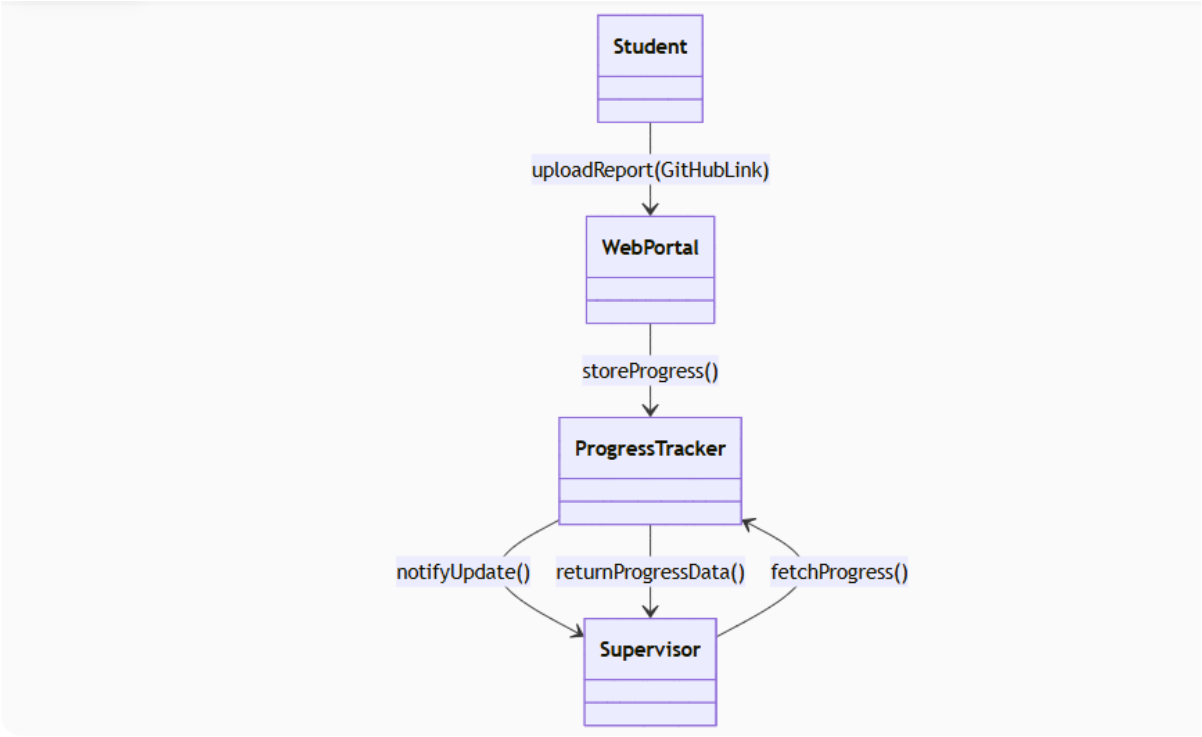


Fig 4.8.1.4. Progress tracking collaboration diagram for student portal

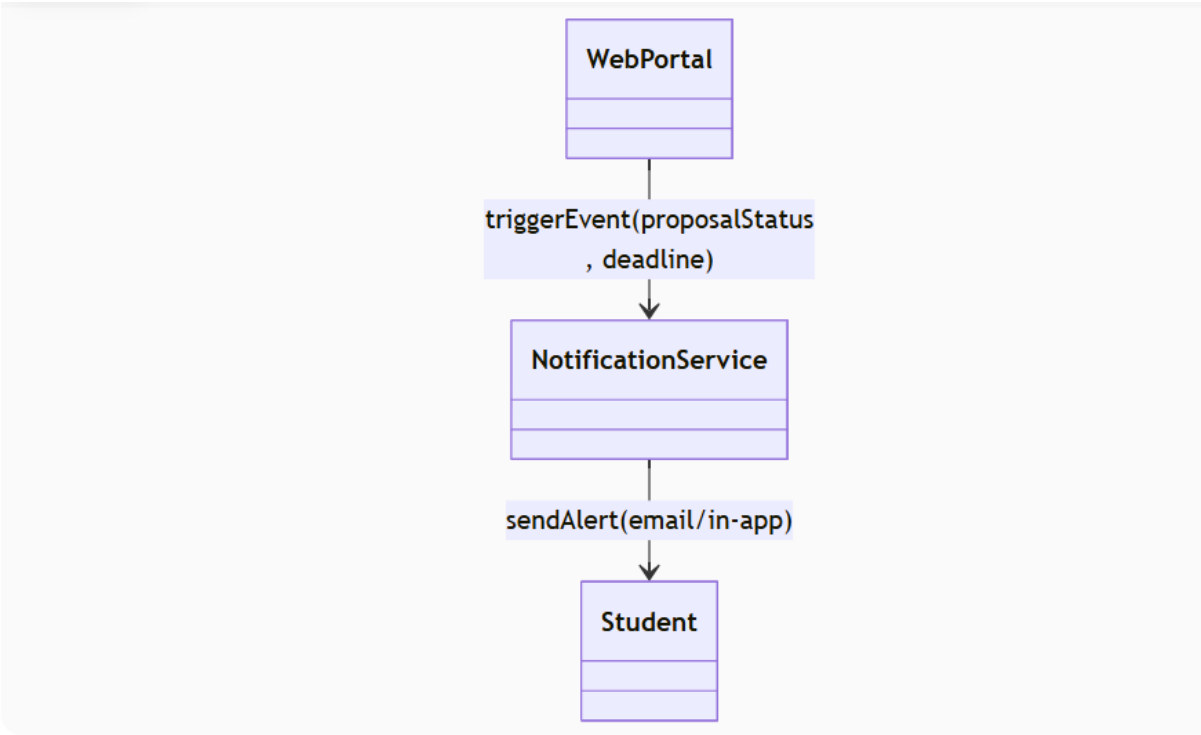


Fig 4.8.1.5. Notifications collaboration diagram for student portal

4.8.2 Supervisor Portal:

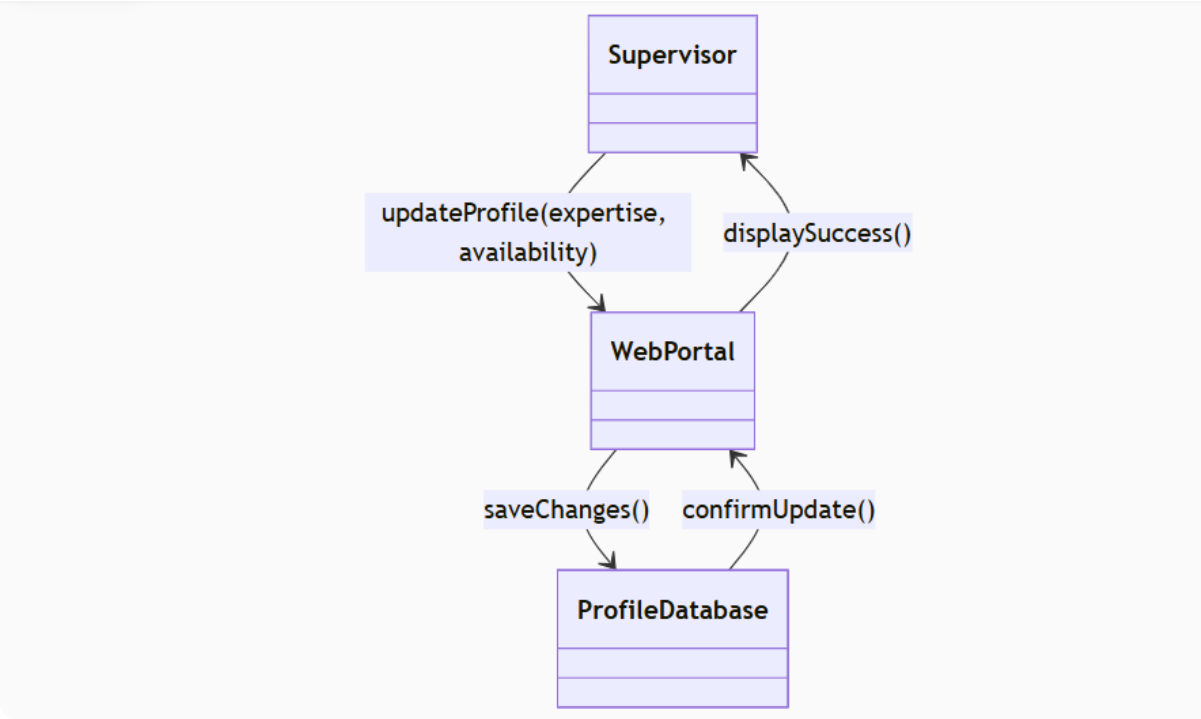


Fig 4.8.2.1. Profile management collaboration diagram for supervisor portal

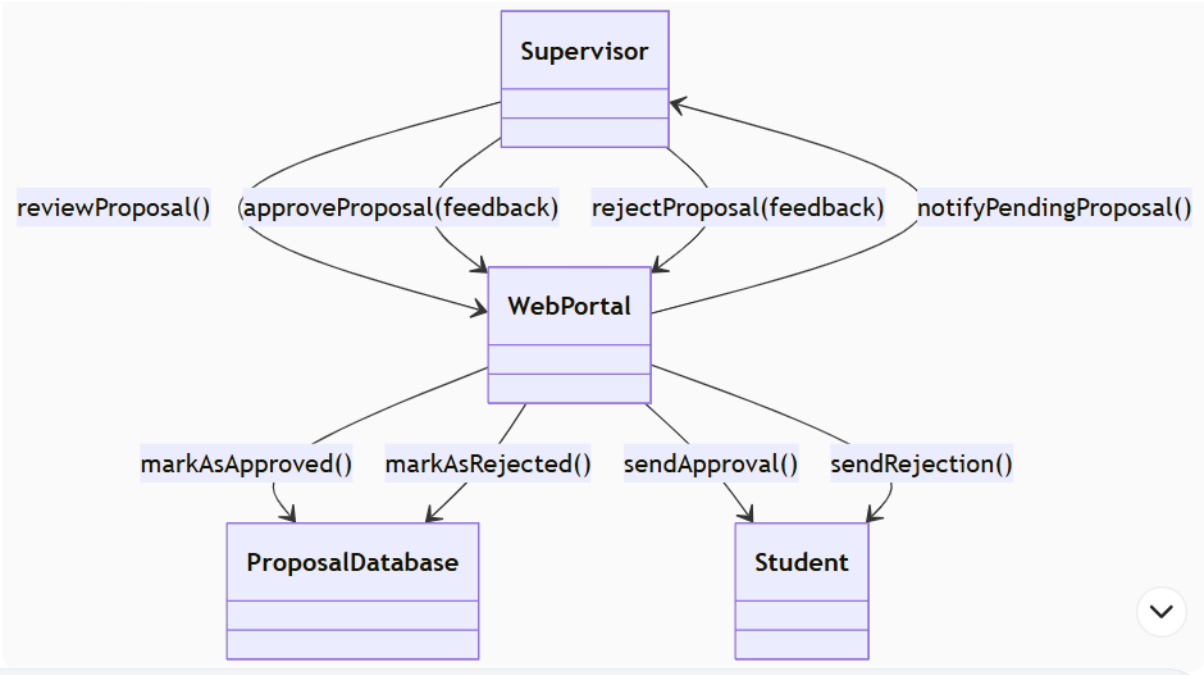


Fig 4.8.2.2. Proposal approval/rejection collaboration diagram for supervisor portal

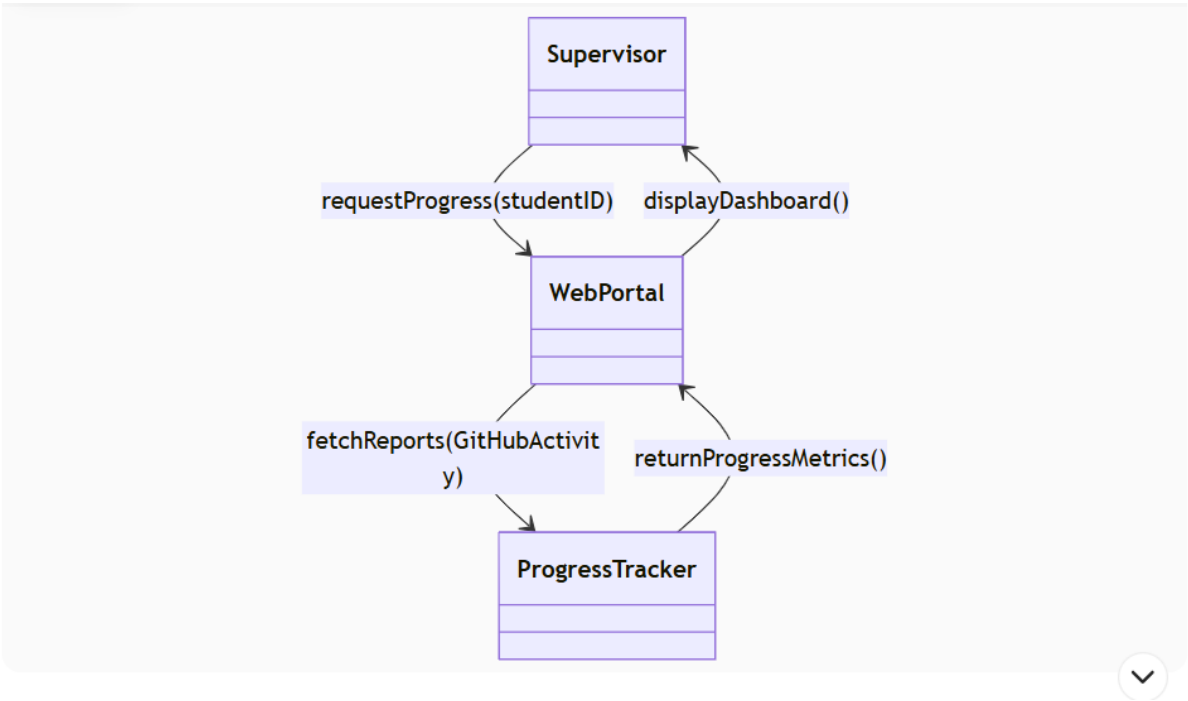


Fig 4.8.2.3. Progress monitoring collaboration diagram for supervisor portal

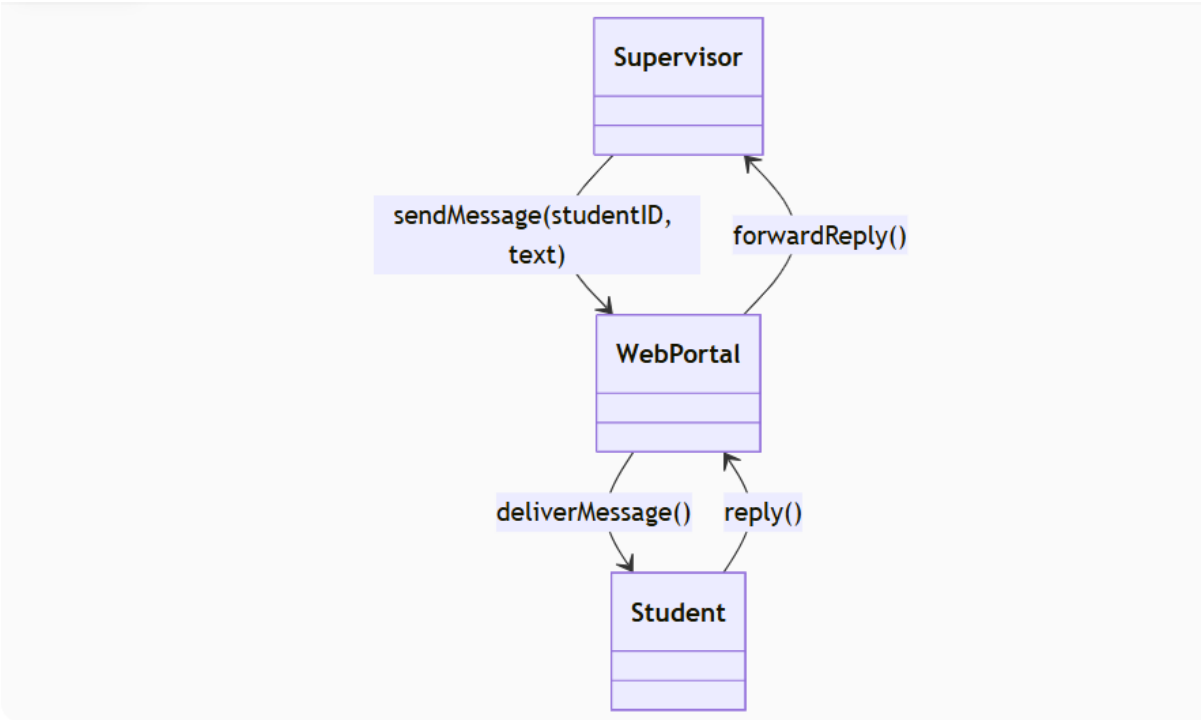


Fig 4.8.2.4. Feedback and communication collaboration diagram for supervisor portal

4.8.3 Coordinator Portal:

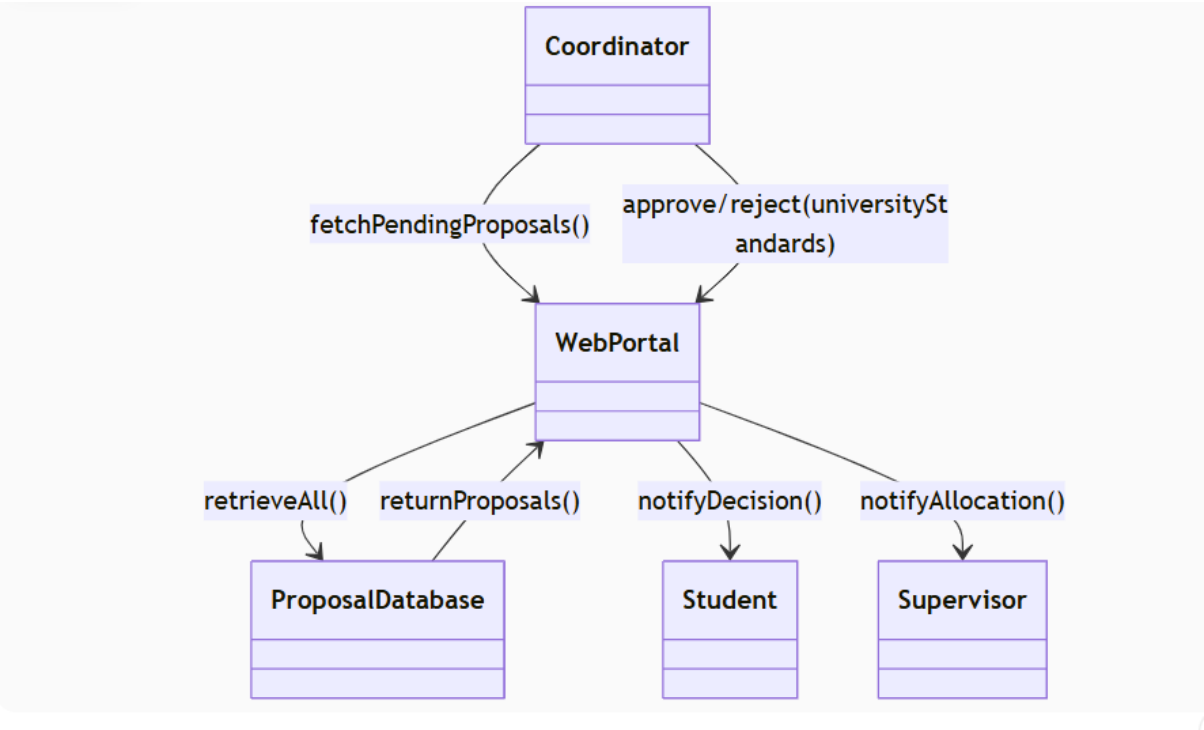


Fig 4.8.3.1. Proposal review collaboration diagram for coordinator portal

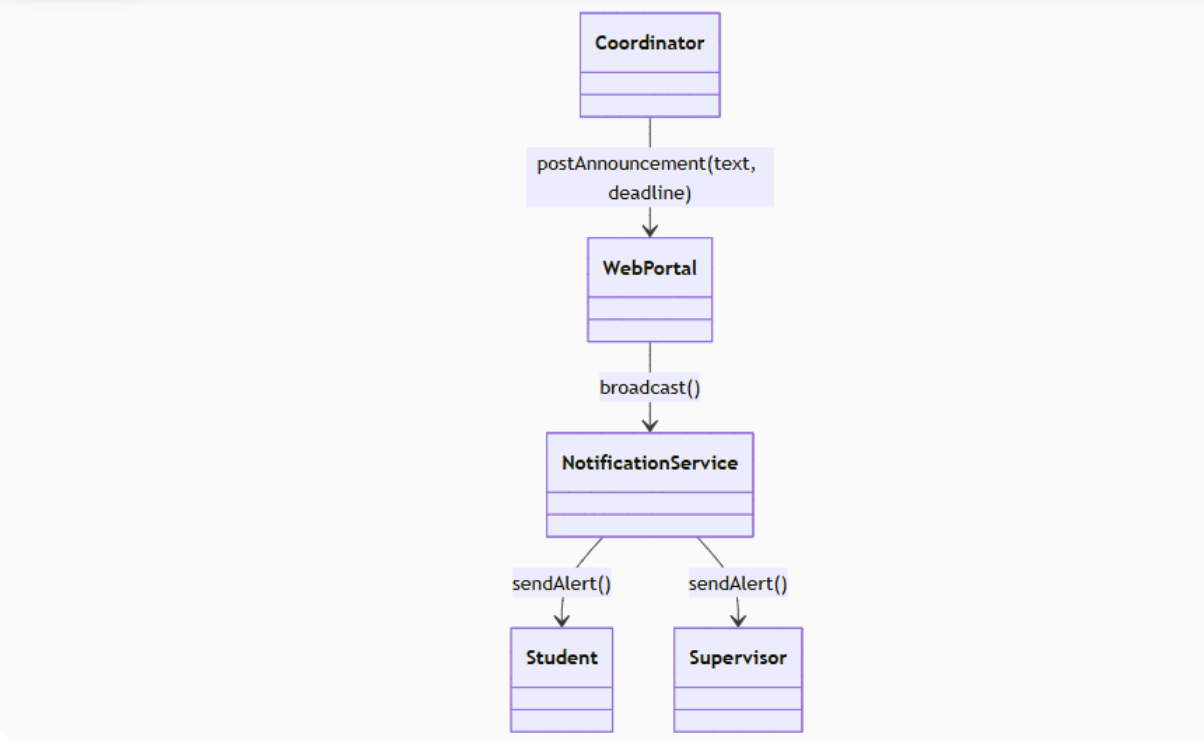


Fig 4.8.3.2. Announcements collaboration diagram for coordinator portal

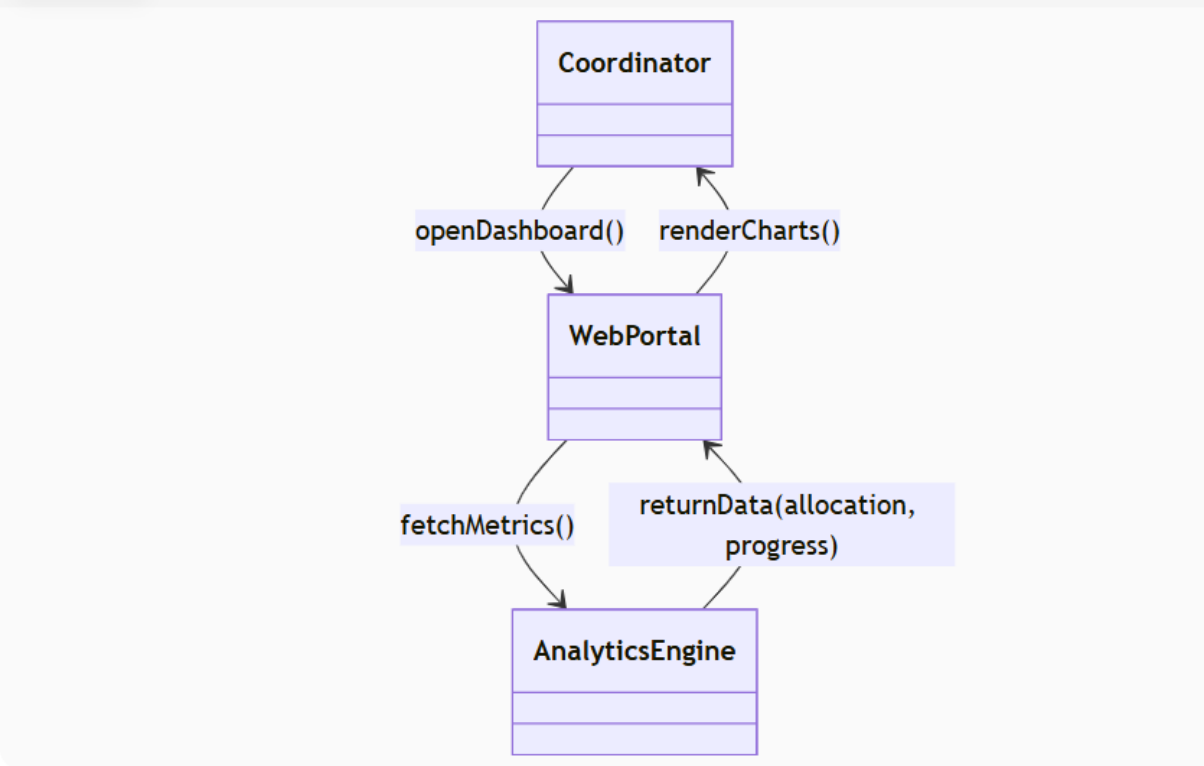


Fig 4.8.3.3. Dashboard collaboration diagram for coordinator portal

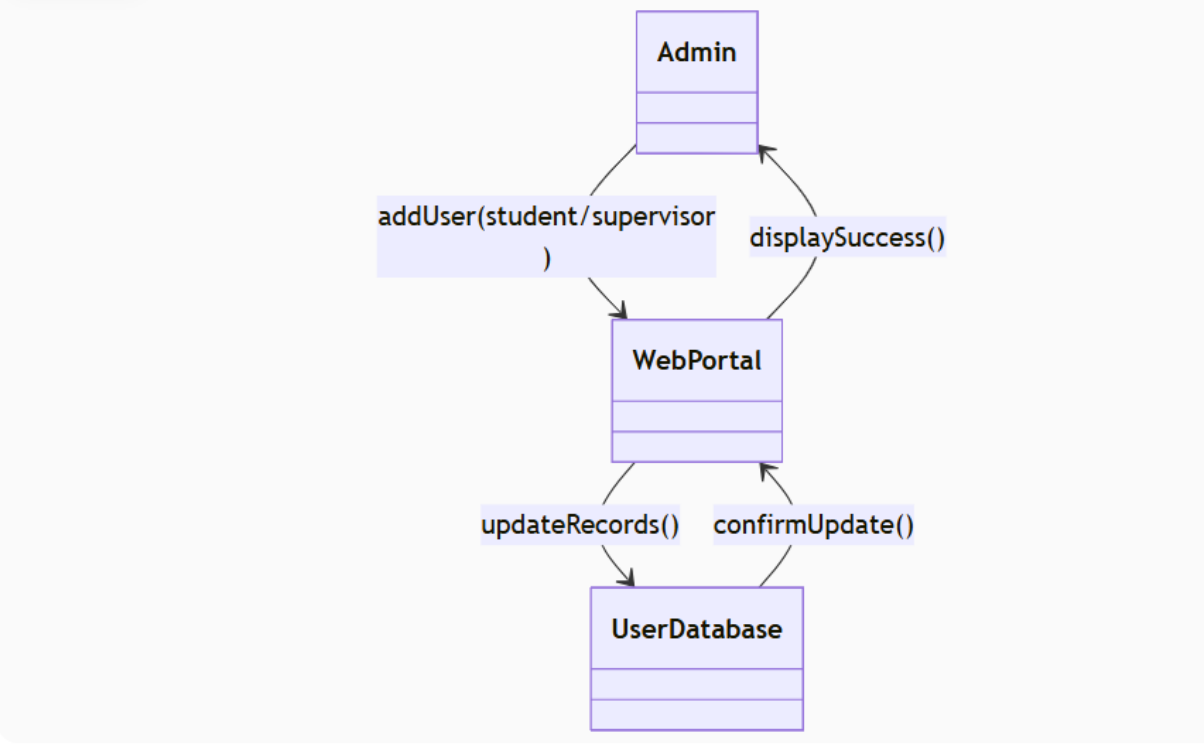


Fig 4.8.3.4. User management collaboration diagram for coordinator portal

4.9 State Transition Diagram

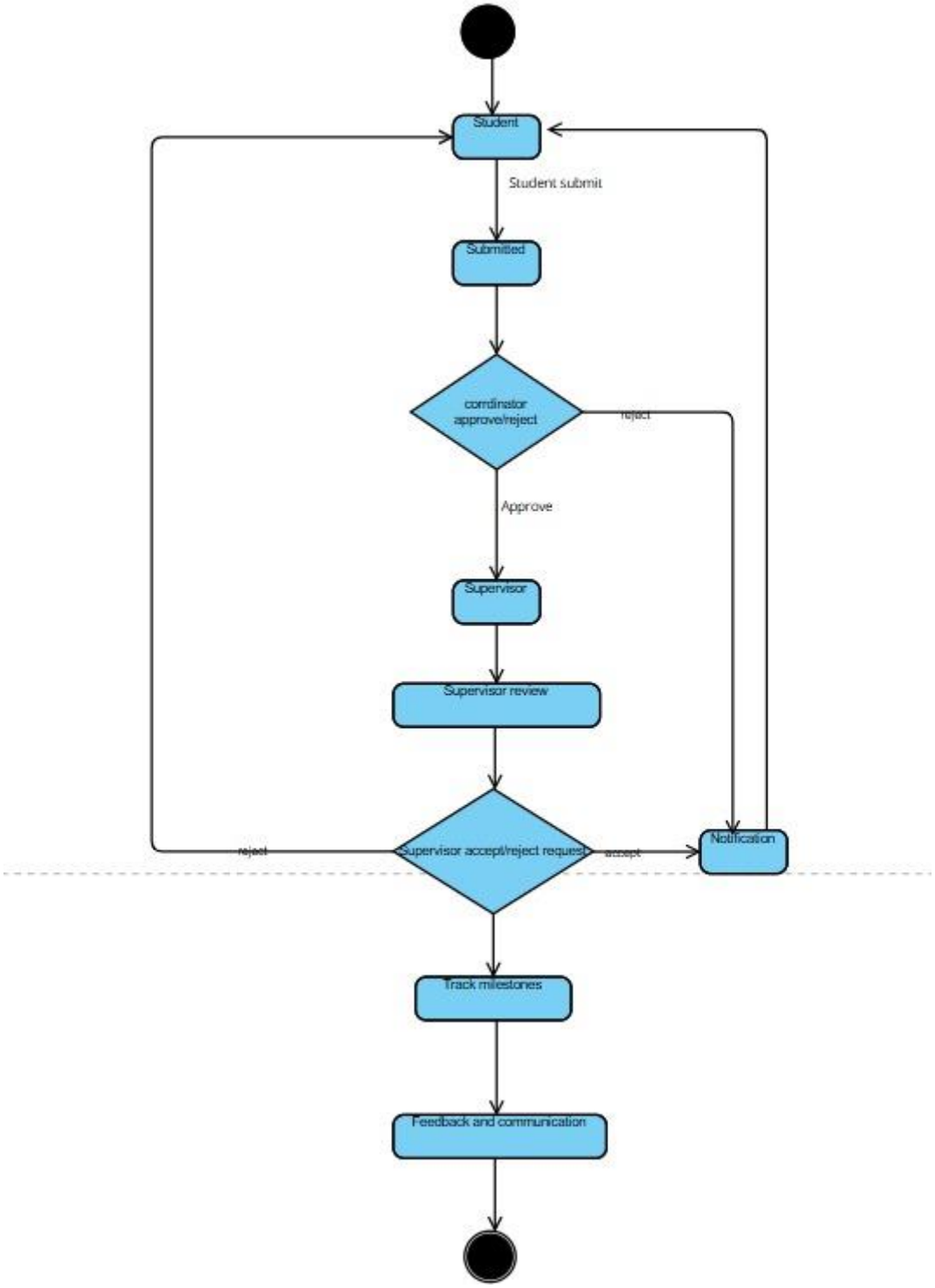


Fig 4.9. State transition diagram of FYPAS

4.10 Component Diagram of System

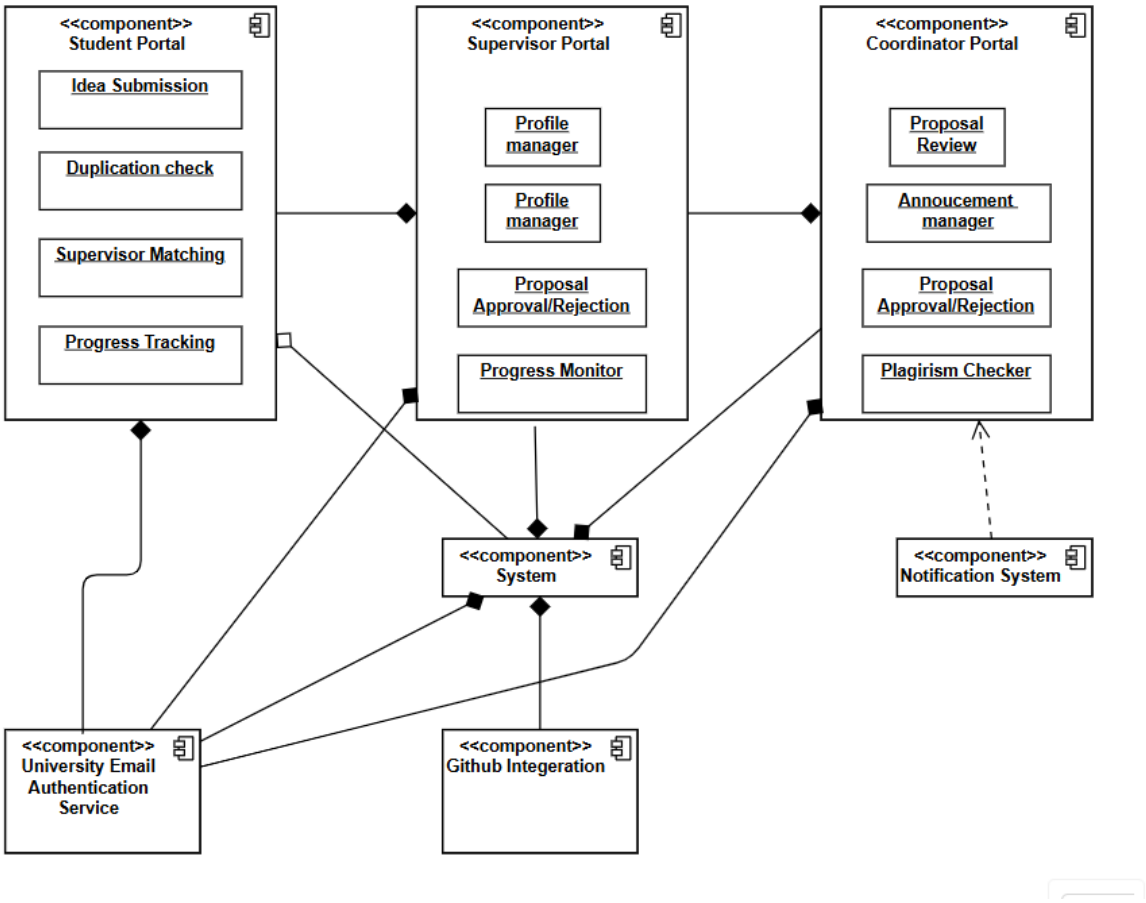


Fig 4.10. Component diagram of FYPAS

4.11 Deployment Diagram

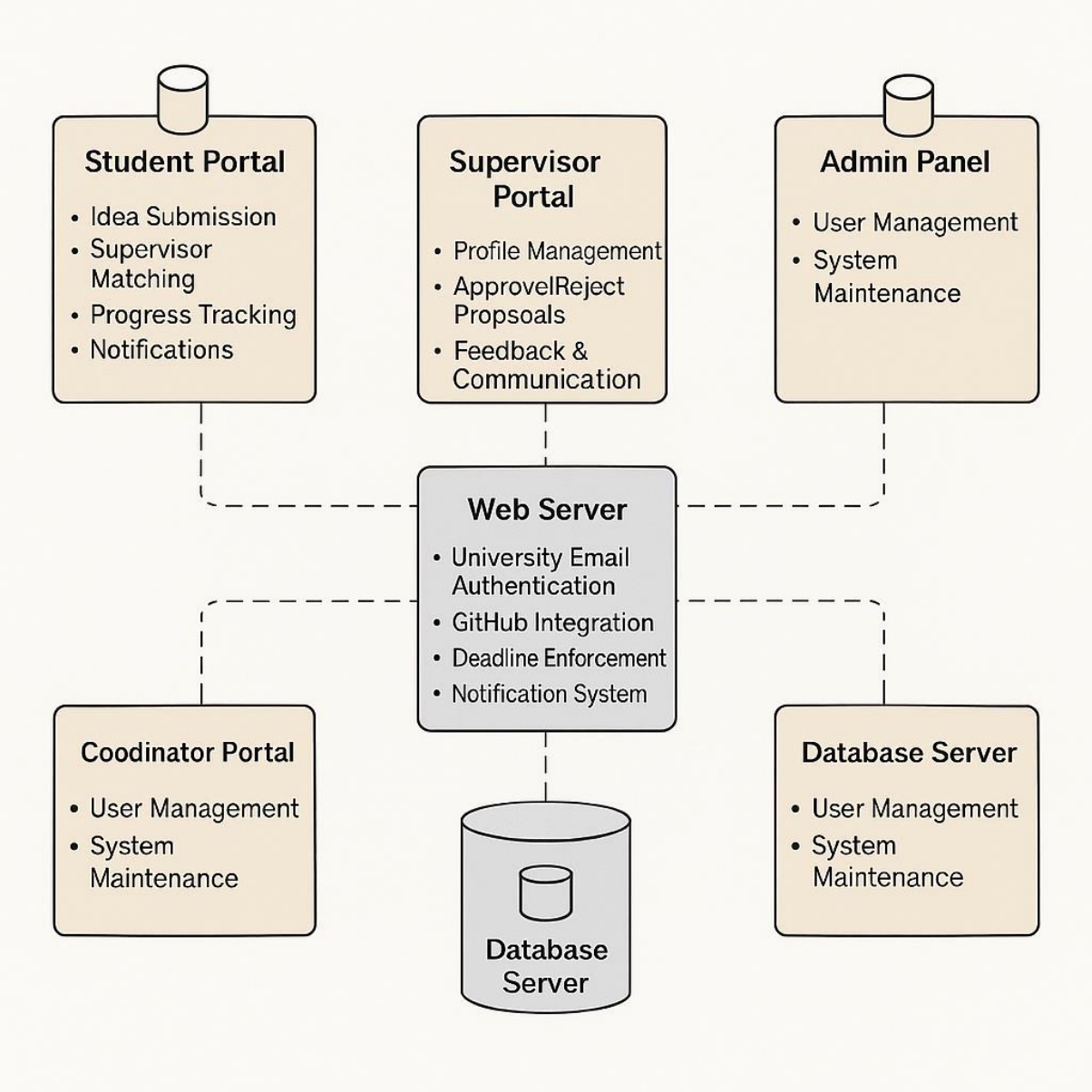


Fig 4.11. Deployment diagram of FYPAS

References:

References

- https://fyp-uol-se.github.io/UoL-SE-FYP/index.html
- https://www.mdpi.com/2305-6703/2/4/31
- https://link.springer.com/chapter/10.1007/978-3-319-72550-5_11
- https://www.sciencedirect.com/science/article/pii/S1877042811011529
- https://www.emerald.com/insight/content/doi/10.1108/jeim-06-2016-0110/full/html